

Nominal α -Unification

Guilherme Borges Brandão[†] Thomas Ammer[‡]
Daniele Nantes Sobrinho[†] Mauricio Ayala-Rinción[†]
Christian Urban[‡] Maribel Fernández[‡]
Mohammad Abdulaziz[‡]

[†] Universidade de Brasília, Brasília D.F., Brazil

[‡] King's College London, London, U.K.

March 10, 2026

Abstract

This theory verifies a non-deterministic procedure for **nominal syntactic unification**, i.e., **nominal α -unification**. The procedure is presented as a set of inductive rules, following Urban's seminal Isabelle formalisation in Isabelle [6, 5].

The formalisation is updated according to the PVS Rocha Oliveira et al. approach [2, 4], by proving separately the properties of symmetry, transitivity, and equivariance of the α -equivalence relation. This treatment of α -equivalence properties was also followed by de Carvalho et al. in the Coq formalisation of nominal α -unification and nominal C-unification [1, 3]. In the PVS formalisation, nominal α -unification is presented as a functional recursive algorithm, proved sound and complete, from which executable code in Lisp can be extracted. In the Coq formalisation, non-deterministic rule-based nominal unification procedures were presented, following Urban's seminal approach, which were proved sound and complete; further, recursive definitions were presented, which were proved equivalent to the inductive procedures, and from which executable code was extracted.

Contents

References

59

theory *Swap*
 imports *Main*
begin

fun *swapa* :: (*string* × *string*) ⇒ *string* ⇒ *string*

```

where
  swapa (b,c) a = (if b = a then c else (if c = a then b else a))

fun swapas:: (string × string) list ⇒ string ⇒ string
  where
    swapas [] a = a
  | swapas (x#pi) a = swapa x (swapas pi a)

lemma [simp]:
  shows swapas [(a,a)] b=b
  by simp

lemma swapas-ab-ba:
  shows swapas [(a,b)] = swapas [(b,a)]
  by auto

lemma swapas-append:
  shows swapas (pi1@pi2) a = swapas pi1 (swapas pi2 a)
  by (induct pi1) auto

lemma swapas-inv [simp]:
  shows swapas (rev pi) (swapas pi a) = a
  apply(induct-tac pi)
  apply(auto simp add: swapas-append)
  done

lemma swapas-rev-pi-a:
  assumes swapas pi a = b
  shows swapas (rev pi) b = a
  using assms by auto

lemma swapas-rev-swaps-id [simp]:
  shows swapas pi (swapas (rev pi) a) = a
  by (metis swapas-rev-pi-a)

lemma swapas-rev-pi-b:
  assumes swapas (rev pi) a = b
  shows swapas pi b = a
  using assms by auto

lemma swapas-comm:
  shows (swapas (pi@[a,b])) c = (swapas ((swapas pi a, swapas pi b)@pi) c)
  by (metis swapa.simps swapas.simps(1,2) swapas-append swapas-rev-pi-a)

end
theory Terms

```

```

imports Swap
begin

```

```

datatype trm = Abst string trm
               | Susp (string × string) list string
               | Unit
               | Atom string
               | Paar trm trm
               | Func string trm

```

```

fun swap :: (string × string) list ⇒ trm ⇒ trm
  where
    swap pi (Unit)           = Unit
  | swap pi (Atom a)         = Atom (swapas pi a)
  | swap pi (Susp pi' X)     = Susp (pi @ pi') X
  | swap pi (Abst a t)       = Abst (swapas pi a) (swap pi t)
  | swap pi (Paar t1 t2)     = Paar (swap pi t1) (swap pi t2)
  | swap pi (Func F t)      = Func F (swap pi t)

```

```

lemma swap-id [simp]:
  shows swap [] t = t
  by (induct-tac t) (auto)

```

```

lemma swap-append:
  shows swap (pi1@pi2) t = swap pi1 (swap pi2 t)
  using swapas-append by (induct t arbitrary: pi1 pi2) auto

```

```

lemma swap-empty:
  assumes swap pi t = Susp [] X
  shows pi = []
  using assms swap.elims by blast

```

```

fun depth :: trm ⇒ nat
  where
    depth (Unit)           = 0
  | depth (Atom a)         = 0
  | depth (Susp pi X)      = 0
  | depth (Abst a t)       = 1 + depth t
  | depth (Func F t)      = 1 + depth t
  | depth (Paar t t')     = 1 + (max (depth t) (depth t'))

```

lemma
Suc-max-left: $\text{Suc}(\max x y) = z \longrightarrow x < z$ **and**
Suc-max-right: $\text{Suc}(\max x y) = z \longrightarrow y < z$
by(*auto*)

lemma *swap-depth* [*simp*]:
shows $\text{depth} (\text{swap } \pi t) = \text{depth } t$
by (*induct t*) (*auto*)

fun *occurs* :: *string* $\Rightarrow$ *trm* $\Rightarrow$ *bool*
where
occurs *X* (*Unit*) = *False*
| *occurs* *X* (*Atom a*) = *False*
| *occurs* *X* (*Susp pi Y*) = (*if X = Y then True else False*)
| *occurs* *X* (*Abst a t*) = *occurs X t*
| *occurs* *X* (*Paar t1 t2*) = (*if (occurs X t1) then True else (occurs X t2)*)
| *occurs* *X* (*Func F t*) = *occurs X t*

fun *vars-trm* :: *trm* $\Rightarrow$ *string set*
where
vars-trm (*Unit*) = {}
| *vars-trm* (*Atom a*) = {}
| *vars-trm* (*Susp pi X*) = {*X*}
| *vars-trm* (*Paar t1 t2*) = (*vars-trm t1*) $\cup$ (*vars-trm t2*)
| *vars-trm* (*Abst a t*) = *vars-trm t*
| *vars-trm* (*Func F t*) = *vars-trm t*

lemma *vars-swap*:
shows $\text{vars-trm} (\text{swap } \pi t) = \text{vars-trm } t$
by (*induct t*) *auto*

fun *sub-trms* :: *trm* $\Rightarrow$ *trm set*
where
sub-trms (*Unit*) = {*Unit*}
| *sub-trms* (*Atom a*) = {*Atom a*}
| *sub-trms* (*Susp pi Y*) = {*Susp pi Y*}
| *sub-trms* (*Abst a t*) = {*Abst a t*} $\cup$ *sub-trms t*
| *sub-trms* (*Paar t1 t2*) = {*Paar t1 t2*} $\cup$ *sub-trms t1* $\cup$ *sub-trms t2*
| *sub-trms* (*Func F t*) = {*Func F t*} $\cup$ *sub-trms t*

fun *psub-trms* :: *trm* $\Rightarrow$ *trm set*
where

```

  psub-trms (Unit)      = {}
| psub-trms (Atom a)    = {}
| psub-trms (Susp pi X) = {}
| psub-trms (Paar t1 t2) = sub-trms t1 ∪ sub-trms t2
| psub-trms (Abst a t)  = sub-trms t
| psub-trms (Func F t)  = sub-trms t

```

```

lemma psub-sub-trms:
  assumes t1 ∈ psub-trms t2
  shows t1 ∈ sub-trms t2
  using assms
  by(induct t2)(auto)

```

```

lemma t-sub-trms-t:
  shows t ∈ sub-trms t
  by (induct t) (auto)

```

```

lemma abst-psub-trms:
  assumes Abst a t1 ∈ sub-trms t2
  shows t1 ∈ psub-trms t2
  using assms
  apply(induct t2 arbitrary: t1)
  apply(auto simp add: t-sub-trms-t intro: psub-sub-trms)
  done

```

```

lemma func-psub-trms:
  assumes Func F t1 ∈ sub-trms t2
  shows t1 ∈ psub-trms t2
  using assms
  apply(induct t2)
  apply(auto simp add: t-sub-trms-t intro: psub-sub-trms)
  done

```

```

lemma paar-psub-trms:
  assumes Paar t1 t2 ∈ sub-trms t3
  shows t1 ∈ psub-trms t3 and t2 ∈ psub-trms t3
  using assms
  apply(induct t3)
  apply(auto simp add: t-sub-trms-t intro: psub-sub-trms)
  done

```

```

lemma t-not-in-psub-trms-t:
  shows t ∉ psub-trms t
  proof(induct t)
  case (Abst x1 t)
  then show ?case

```

```

      using abst-psub-trms by auto
next
  case (Susp x1 x2)
  then show ?case by simp
next
  case Unit
  then show ?case by simp
next
  case (Atom x)
  then show ?case by simp
next
  case (Paar t1 t2)
  then have Paar t1 t2  $\notin$  sub-trms t1 Paar t1 t2  $\notin$  sub-trms t2
    using paar-psub-trms by auto
  then show ?case by simp
next
  case (Func f t)
  then show ?case
    using func-psub-trms by auto
qed

```

```

lemma if-sub-not-eq-then-psub:
  assumes t1  $\in$  sub-trms t2 t1  $\neq$  t2
  shows t1  $\in$  psub-trms t2
  using assms
  by (induct t2) auto

```

```

lemma depth-psub-trms:
  assumes t1  $\in$  psub-trms t2
  shows depth t1 < depth t2
  using assms
proof(induct t2 arbitrary: t1)
  case (Abst a t)
  then show ?case
    using depth.simps(4)[of a t]
      psub-trms.simps(5)[of a t] if-sub-not-eq-then-psub
    by fastforce
next
  case (Susp pi X)
  then show ?case by simp
next
  case Unit
  then show ?case by simp
next
  case (Atom a)
  then show ?case by simp
next
  case (Paar t1' t2')

```

```

    then show ?case
      using depth.simps(6)[of t1' t2']
        psub-trms.simps(4) if-sub-not-eq-then-psub by fastforce
  next
    case (Func f t1')
    then show ?case
      using depth.simps(5)[of f t1']
        psub-trms.simps(6)[of f t1'] if-sub-not-eq-then-psub
        by fastforce
qed

end

theory Atoms

imports Terms

begin

fun atms :: (string × string) list ⇒ string set where
  atms [] = {} |
  atms (x#xs) = ((atms xs) ∪ {fst(x),snd(x)})

lemma [simp]:
  atms (xs@ys) = atms xs ∪ atms ys
  by (induct xs) auto

lemma [simp]:
  atms (rev pi) = atms pi
  by (induct pi) auto

lemma a-not-in-atms:
  assumes a ∉ atms pi
  shows a = swapas pi a
  using assms by (induct pi) auto

lemma swapas-pi-ineq-a:
  assumes swapas pi a ≠ a
  shows a ∈ atms pi
  using a-not-in-atms assms by force

lemma a-ineq-swapas-pi:
  assumes a ≠ swapas pi a
  shows a ∈ atms pi

```

```

using a-not-in-atms assms by blast

lemma swapas-pi-in-atms:
  assumes  $a \in \text{atms } \pi$ 
  shows  $\text{swapas } \pi \ a \in \text{atms } \pi$ 
  using assms by (metis a-not-in-atms swaps-rev-pi-a)

end

theory Disagreement

imports Atoms

begin

definition ds ::  $(\text{string} \times \text{string}) \text{ list} \Rightarrow (\text{string} \times \text{string}) \text{ list} \Rightarrow \text{string set}$  where
  ds-def:  $\text{ds } xs \ ys \equiv \{ a \mid a \in (\text{atms } xs \cup \text{atms } ys) \wedge (\text{swapas } xs \ a \neq \text{swapas } ys \ a) \}$ 

lemma
  ds-elem:  $\llbracket \text{swapas } \pi \ a \neq a \rrbracket \Longrightarrow a \in \text{ds } \pi$ 
  using ds-def swaps-pi-ineq-a by simp

corollary ds-elem-cp:  $a \notin \text{ds } \pi \Longrightarrow \text{swapas } \pi \ a = a$ 
  using ds-elem by blast

lemma
  elem-ds:  $\llbracket a \in \text{ds } \pi \rrbracket \Longrightarrow a \neq \text{swapas } \pi \ a$ 
  using ds-def by simp

lemma
  ds-sym:  $\text{ds } \pi_1 \ \pi_2 = \text{ds } \pi_2 \ \pi_1$ 
  using ds-def by auto

lemma
  ds-trans:  $c \in \text{ds } \pi_1 \ \pi_3 \Longrightarrow (c \in \text{ds } \pi_1 \ \pi_2 \vee c \in \text{ds } \pi_2 \ \pi_3)$ 
  using ds-def a-not-in-atms swaps-pi-ineq-a by auto

```


lemma *ds-cancel-pi-left*:
assumes $(c \in ds \ (pi1@pi) \ (pi2@pi))$
shows $(swapas \ pi \ c \in ds \ pi1 \ pi2)$
using *assms ds-def swapas-append a-ineq-swapas-pi a-not-in-atms*
by (*metis (mono-tags, lifting) Un-iff mem-Collect-eq*)

lemma *ds-cancel-pi-right*:
 $(swapas \ pi \ c \in ds \ pi1 \ pi2) \implies (c \in ds \ (pi1@pi) \ (pi2@pi))$
apply(*simp only: ds-def*)
apply(*auto*)
apply(*simp-all add: swapas-append*)
apply(*rule a-ineq-swapas-pi, clarify,*
drule a-not-in-atms, drule a-not-in-atms, simp) +
done

lemma *ds-equality*:
 $(ds \ [] \ pi) - \{a, swapas \ pi \ a\} = (ds \ [] \ ((a, swapas \ pi \ a) \# pi)) - \{swapas \ pi \ a\}$
using *ds-def by auto*

lemma *ds-7*:
 $\llbracket b \neq swapas \ pi \ b; a \in ds \ [] \ ((b, swapas \ pi \ b) \# pi) \rrbracket \implies a \in ds \ [] \ pi$
using *ds-def swapas-pi-in-atms a-ineq-swapas-pi swapas-rev-pi-a*
ds-elem elem-ds swapa.simps swapas.simps(2)
by *metis*

lemma *ds-cancel-pi-front*:
 $ds \ (pi@pi1) \ (pi@pi2) = ds \ pi1 \ pi2$
apply(*simp only: ds-def*)
apply(*auto*)
apply(*simp-all add: swapas-append*)
apply(*rule swapas-pi-ineq-a, clarify, drule a-not-in-atms, simp*) +
apply(*drule swapas-rev-pi-a, simp*) +
done

lemma *ds-rev-pi-pi*:
 $ds \ ((rev \ pi1)@pi1) \ pi2 = ds \ [] \ pi2$
apply(*simp only: ds-def*)
apply(*auto*)
apply(*simp-all add: swapas-append*)
apply(*drule a-ineq-swapas-pi, assumption*) +
done

```

lemma ds-rev:
   $ds \ [] \ ((rev \ pi1) @ pi2) = ds \ pi1 \ pi2$ 
  using ds-cancel-pi-front ds-rev-pi-pi by blast

lemma ds-acabbc:
   $\llbracket a \neq b; b \neq c; c \neq a \rrbracket \implies ds \ [(a, b), (b, c)] \ [(a, c)] = \{a, b\}$ 
  using ds-def by auto

lemma ds-baab:
   $\llbracket a \neq b \rrbracket \implies ds \ [(b, a)] \ [(a, b)] = \{\}$ 
  using ds-def by auto

lemma ds-baab-id:
   $\llbracket a \neq b \rrbracket \implies ds \ ([ (b, a) ] @ [ (a, b) ]) \ [] = \{\}$ 
  using ds-def ds-rev ds-baab by simp

lemma ds-abab:
   $\llbracket a \neq b \rrbracket \implies ds \ [] \ [(a, b), (a, b)] = \{\}$ 
  using ds-def by auto

lemma ds-comm:
   $ds \ (pi @ [(a, b)]) \ ([ (swapas \ pi \ a, \ swaps \ pi \ b) ] @ pi) = \{\}$ 
  using swapas-comm ds-def by simp

lemma ds-rev-pi-id:
   $ds \ (rev \ pi @ pi) \ [] = \{\}$ 
  using ds-rev-pi-pi[of pi <[]>] elem-ds
  ds-sym[of <(rev pi @ pi)> <[]>] by fastforce

lemma ds-pi-rev-id:
   $ds \ (pi @ rev \ pi) \ [] = \{\}$ 
  using ds-rev-pi-id[of <rev pi>] rev-rev-ident[of pi]
  by simp

lemma ds-swapas-eq:
   $ds \ pi1 \ pi2 = \{\} \implies swaps \ pi1 \ a = swaps \ pi2 \ a$ 
  using ds-elem[of <rev pi1 @ pi2>] ds-rev[of pi1 pi2]
  empty-iff[of a] swaps-append[of <rev pi1> pi2 a] swaps-inv by metis

fun flatten ::  $(string \times string)list \Rightarrow string \ list$  where
  flatten [] = [] |
  flatten (x#xs) = (fst x)#(snd x)#(flatten xs)

definition ds-list ::  $(string \times string)list \Rightarrow (string \times string)list \Rightarrow string \ list$ 
where

```

ds-list-def: $ds\text{-}list\ pi1\ pi2 \equiv remdups\ ([x.\ x <- (flatten\ (pi1@pi2)),\ x \in ds\ pi1\ pi2])$

lemma *set-flatten-eq-atms*:
 $set\ (flatten\ pi) = atms\ pi$
by (*induct pi*) *auto*

lemma *ds-list-eq-ds*:
 $set\ (ds\text{-}list\ pi1\ pi2) = ds\ pi1\ pi2$
using *ds-list-def ds-def set-flatten-eq-atms* **by** *auto*

end
theory *Fresh*

imports *Disagreement*

begin

type-synonym *fresh-envs* = (*string* $\times$ *string*) *set*

inductive *fresh* :: *fresh-envs* $\Rightarrow$ *string* $\Rightarrow$ *trm* $\Rightarrow$ *bool* (*-* $\vdash$ *-* $\#$ - [*80,80,80*] *80*)

where

fresh-abst-ab[*intro!*]: $\llbracket nabra \vdash a \# t; a \neq b \rrbracket \Longrightarrow nabra \vdash a \# Abst\ b\ t \mid$
fresh-abst-aa[*intro!*]: $nabra \vdash a \# Abst\ a\ t \mid$
fresh-unit[*intro!*]: $nabra \vdash a \# Unit \mid$
fresh-atom[*intro!*]: $a \neq b \Longrightarrow nabra \vdash a \# Atom\ b \mid$
fresh-susp[*intro!*]: $(swapas\ (rev\ pi)\ a, X) \in nabra \Longrightarrow nabra \vdash a \# Susp\ pi\ X \mid$
fresh-paar[*intro!*]: $\llbracket nabra \vdash a \# t1; nabra \vdash a \# t2 \rrbracket \Longrightarrow nabra \vdash a \# Paar\ t1\ t2 \mid$
fresh-func[*intro!*]: $nabra \vdash a \# t \Longrightarrow nabra \vdash a \# Func\ F\ t$

inductive-cases *Fresh-elim*s:

$nabra \vdash a \# Abst\ b\ t$
 $nabra \vdash a \# Unit$
 $nabra \vdash a \# Atom\ b$
 $nabra \vdash a \# Susp\ pi\ X$
 $nabra \vdash a \# Paar\ t1\ t2$
 $nabra \vdash a \# Func\ F\ t$

lemma *fresh-swap-eqvt*:
assumes $nabra \vdash a \# t$
shows $nabra \vdash swapas\ pi\ a \# swap\ pi\ t$
using *assms*
by (*induct arbitrary: pi rule: fresh.induct*)
(*auto simp add: swapas-append dest: swapas-rev-pi-a*)

lemma *fresh-swap-left*:
assumes $nabla \vdash a \# \text{swap } \pi \ t$
shows $nabla \vdash \text{swapas } (\text{rev } \pi) \ a \# \ t$
using *assms*
proof(*induct t arbitrary: a pi*)
case (*Abst x1 t*)
have $nabla \vdash a \# \text{swap } \pi \ (\text{Abst } x1 \ t)$ **by** *fact*
then have $(nabla \vdash a \# \text{swap } \pi \ t \wedge a \neq \text{swapas } \pi \ x1) \vee (a = \text{swapas } \pi \ x1)$
by (*metis Fresh-elim(1) swap.simps(4)*)
moreover {
assume *as*: $nabla \vdash a \# \text{swap } \pi \ t \wedge a \neq \text{swapas } \pi \ x1$
have $nabla \vdash \text{swapas } (\text{rev } \pi) \ a \# \text{Abst } x1 \ t$
using *Abst.hyps as* **by** *auto*
}
moreover {
assume *as*: $a = \text{swapas } \pi \ x1$
then have $nabla \vdash \text{swapas } (\text{rev } \pi) \ a \# \text{Abst } x1 \ t$
by (*simp add: fresh-abst-aa*)
}
ultimately show $nabla \vdash \text{swapas } (\text{rev } \pi) \ a \# \text{Abst } x1 \ t$
by *argo*
next
case (*Susp x1 x2*)
have $nabla \vdash a \# \text{swap } \pi \ (\text{Susp } x1 \ x2)$ **by** *fact*
have $(\text{swapas } (\text{rev } x1) \ (\text{swapas } (\text{rev } \pi) \ a), x2) \in nabla$
by (*metis Fresh-elim(4) Susp rev-append swap.simps(3) swapas-append*)
then show $nabla \vdash \text{swapas } (\text{rev } \pi) \ a \# \text{Susp } x1 \ x2$
by (*simp add: fresh-susp*)
next
case (*Atom x*)
have $nabla \vdash a \# \text{swap } \pi \ (\text{Atom } x)$ **by** *fact*
then have $\text{swapas } \pi \ x \neq a$ **by** (*auto elim: Fresh-elim*)
then have $x \neq \text{swapas } (\text{rev } \pi) \ a$ **using** *swapas-rev-pi-b* **by** *blast*
then show $nabla \vdash \text{swapas } (\text{rev } \pi) \ a \# \text{Atom } x$
by (*simp add: fresh-atom*)
qed (*auto elim: Fresh-elim*)

lemma *fresh-swap-right*:
assumes $nabla \vdash \text{swapas } (\text{rev } \pi) \ a \# \ t$
shows $nabla \vdash a \# \text{swap } \pi \ t$
using *assms fresh-swap-eqvt*[*of nabla <swapas (rev pi) a> t pi*]
swapas-rev-swapas-id[*of pi a*] **by** *simp*

lemma *fresh-weak*:
assumes $nabla1 \vdash a \# \ t$
shows $(nabla1 \cup nabla2) \vdash a \# \ t$
using *assms*

```

by(induct rule: fresh.induct)(auto)

lemma ds-empty-fresh-1:
  assumes ds pi1 pi2 = {}
  shows nabl a ⊢ swapi a # t ⇒ nabl a ⊢ swapi pi2 a # t
  using asms ds-swapi-eq by simp

lemma ds-empty-fresh-2:
  assumes ds pi1 pi2 = {}
  shows nabl a ⊢ a # swapi pi1 t ⇒ nabl a ⊢ a # swapi pi2 t
  using asms
proof-
  assume assm1: nabl a ⊢ a # swapi pi1 t
  have i: swapi (rev pi1) a = swapi (rev pi2) a
    using asms ds-swapi-eq swapi-rev-pi-a by metis
  with asms have ii: nabl a ⊢ swapi (rev pi2) a # t
    using fresh-swap-left[OF assm1] i by simp
  show ?thesis
    using fresh-swap-right[OF ii] by simp
qed

end
theory PreEqu

imports Fresh

begin

inductive equ :: fresh-envs ⇒ trm ⇒ trm ⇒ bool ( - ⊢ - ≈ - [80,80,80] 80) where
  equ-abst-ab[intro!]: [[a≠b;(nabl a ⊢ a # t2);(nabl a ⊢ t1 ≈ (swapi [(a,b)] t2))]]
    ⇒ (nabl a ⊢ Abst a t1 ≈ Abst b t2) |
  equ-abst-aa[intro!]: (nabl a ⊢ t1 ≈ t2) ⇒ (nabl a ⊢ Abst a t1 ≈ Abst a t2) |
  equ-unit[intro!]: (nabl a ⊢ Unit ≈ Unit) |
  equ-atom[intro!]: a=b ⇒ nabl a ⊢ Atom a ≈ Atom b |
  equ-susp[intro!]: (∀ c ∈ ds pi1 pi2. (c,X) ∈ nabl a) ⇒ (nabl a ⊢ Susp pi1 X ≈
    Susp pi2 X) |
  equ-paar[intro!]: [[(nabl a ⊢ t1≈t2);(nabl a ⊢ s1≈s2)]] ⇒ (nabl a ⊢ Paar t1 s1 ≈
    Paar t2 s2) |
  equ-func[intro!]: (nabl a ⊢ t1 ≈ t2) ⇒ (nabl a ⊢ Func F t1 ≈ Func F t2)

inductive-cases Equ-elim:
  nabl a ⊢ Atom a ≈ Atom b
  nabl a ⊢ Unit ≈ Unit
  nabl a ⊢ Susp pi1 X ≈ Susp pi2 X
  nabl a ⊢ Paar s1 t1 ≈ Paar s2 t2
  nabl a ⊢ Func F t1 ≈ Func F t2
  nabl a ⊢ Abst a t1 ≈ Abst a t2
  nabl a ⊢ Abst a t1 ≈ Abst b t2

```

```

lemma equ-depth:
  assumes  $\text{nabla} \vdash t1 \approx t2$ 
  shows  $\text{depth } t1 = \text{depth } t2$ 
  using assms by (rule equ.induct, simp-all)

lemma rev-pi-pi-equ:  $(\text{nabla} \vdash \text{swap } (\text{rev } \pi) (\text{swap } \pi t) \approx t)$ 
proof(induct t)
  case (Susp  $\pi' X$ )
  then show ?case
    using ds-cancel-pi-left[of -  $\langle \text{rev } \pi @ \pi \rangle$  -  $\langle [] \rangle$ ]
    ds-rev-pi-pi elem-ds ds-rev-pi-id by auto
qed (auto)

lemma equ-pi-right:
  assumes  $\forall a \in \text{ds } [] \pi. \text{nabla} \vdash a \# t$ 
  shows  $\text{nabla} \vdash t \approx \text{swap } \pi t$ 
  using assms
proof(induct t arbitrary: pi)
  case (Abst  $b t'$ )
  have  $\text{swapas } \pi b = b \vee \text{swapas } \pi b \neq b$  by simp
  moreover
  {
    assume eq:  $\text{swapas } \pi b = b$ 
    have  $\text{nabla} \vdash \text{Abst } b t' \approx \text{Abst } b (\text{swap } \pi t')$ 
      apply (rule equ-abst-aa)
      apply (rule Abst.hyps)
      apply (rule ballI)
    subgoal for a
      apply (rule Fresh-elim1)[of  $\text{nabla } a b t'$ ]
      using Abst.prem1 elem-ds[of  $a \pi$ ] eq by auto
    done
    then have  $\text{nabla} \vdash \text{Abst } b t' \approx \text{swap } \pi (\text{Abst } b t')$  using eq by simp
  }
  moreover
  {
    assume neg:  $b \neq \text{swapas } \pi b$ 
    obtain c where c-def:  $c = \text{swapas } \pi b$  by simp
    have  $c \in \text{ds } [] \pi$ 
      by (metis append.right-neutral c-def ds-cancel-pi-front ds-cancel-pi-left ds-elem
neg)
    then have one:  $\text{nabla} \vdash c \# \text{Abst } b t'$  using assms Abst.prem1 by auto
    have two:  $b \neq c$  using neg c-def by blast
    have  $\text{nabla} \vdash c \# t'$  using one two by cases auto
    have  $\text{nabla} \vdash \text{Abst } b t' \approx \text{Abst } c (\text{swap } \pi t')$ 
      proof(rule equ-abst-ab)
        show  $b \neq c$  using neg c-def by blast
        have b-is-swap:  $b = \text{swapas } (\text{rev } \pi) c$  using c-def by simp

```

```

    show  $nabla \vdash b \# \text{swap } \pi \ t'$ 
    using Abst.prems Fresh-elims(1) ds-elem fresh-swap-right neq swapas-rev-pi-a
  by metis
    show  $nabla \vdash t' \approx \text{swap } [(b, c)] (\text{swap } \pi \ t')$ 
    proof -
      have fresh-ext:
         $\forall a \in ds \ []. ((b, c) \# \pi). \nabla \vdash a \# t'$ 
      proof (rule ballI)
        fix a assume A:  $a \in ds \ []. ((b, c) \# \pi)$ 
        then have  $a = c \vee a \in ds \ [] \ \pi$ 
          using c-def ds-7 neq by auto
        then show  $\nabla \vdash a \# t'$ 
        proof
          assume  $a = c$ 
          with  $\langle \nabla \vdash c \# t' \rangle$  show ?thesis by simp
        next
          assume  $a \in ds \ [] \ \pi$ 
          show ?thesis
          proof (rule Fresh-elims(1)[of  $\nabla \vdash a \ b \ t'$ ], goal-cases)
            case 1
            then show ?case
              using Abst.prems  $\langle a \in ds \ [] \ \pi \rangle$ 
              by simp
            next
              case 2
              then show ?case by simp
            next
              case 3
              have  $a \neq \text{swapas } ((b, c) \# \pi) \ a$ 
                using A elem-ds
                by blast
              hence  $a \neq b$  using c-def A
                by (auto simp add: if-split)
              hence False using 3 by simp
              then show ?case by simp
          qed
        qed
      qed
    from Abst.hyps[OF fresh-ext]
    have  $\nabla \vdash t' \approx \text{swap } [(b, c)] @ \pi \ t'$ 
      by simp
    moreover have  $\text{swap } [(b, c)] @ \pi \ t' = \text{swap } [(b, c)] (\text{swap } \pi \ t')$ 
      using swap-append by blast
    ultimately show ?thesis by simp
  qed
qed
then have  $\nabla \vdash \text{Abst } b \ t' \approx \text{swap } \pi (\text{Abst } b \ t')$  using neq c-def by force
}

```

```

ultimately show ?case by argo

next
case (Susp pi' X)
have  $\forall a \in ds \ []. \text{ pi. } \text{nabla} \vdash a \# \text{ Susp } \text{pi}' X$  by fact
then have  $\text{nabla} \vdash \text{ Susp } \text{pi}' X \approx \text{ Susp } (\text{pi} @ \text{pi}') X$ 
  using Fresh-elim(4) equ-susp ds-def ds-cancel-pi-left
  by (metis (lifting) append-self-conv2 swapas-rev-pi-a)
then show ?case by simp
next
case Unit
then show ?case
  using equ-unit swap.simps(2) by force
next
case (Atom b)
then show ?case
  apply simp
  apply (rule equ-atom)
  using Fresh-elim(3) ds-elem-cp
  by metis
next
case (Paar t1 t2)
then have hypsp1:  $(\forall a \in ds \ []. \text{ pi. } \text{nabla} \vdash a \# t1)$ 
  and hypsp2:  $(\forall a \in ds \ []. \text{ pi. } \text{nabla} \vdash a \# t2)$ 
  using Paar.premis Fresh-elim(5)
  apply meson
  using Paar.premis Fresh-elim(5) by blast
have  $\text{nabla} \vdash \text{ Paar } t1 t2 \approx \text{ Paar } (\text{swap } \text{pi } t1) (\text{swap } \text{pi } t2)$ 
  apply (rule equ-paar)
  apply (rule Paar.hyps)
  apply (simp add: hypsp1)
  by (simp add: Paar.hyps(2) hypsp2)
then show ?case by simp
next
case (Func f t')
then have hypsf:  $\forall a \in ds \ []. \text{ pi. } \text{nabla} \vdash a \# t'$  using fresh-func
  by (metis Fresh-elim(6))
have  $\text{nabla} \vdash \text{ Func } f t' \approx \text{ Func } f (\text{swap } \text{pi } t')$ 
  apply (rule equ-func)
  apply (rule Func.hyps)
  using hypsf by auto
then show ?case
  by simp
qed

lemma pi-comm:  $\text{nabla} \vdash (\text{swap } (\text{pi} @ [(a,b)]) t) \approx (\text{swap } ((\text{swapas } \text{pi } a, \text{swapas } \text{pi } b)) @ \text{pi}) t$ 
proof (induct t)
case (Abst c t)

```



```

    then show ?case using swapas-comm by force
next
  case (Susp  $\pi'$   $X$ )
  have rew1:
     $\text{swap } (pi \text{ @ } [(a,b)]) (Susp \pi' X) = Susp (pi \text{ @ } [(a,b)] \text{ @ } \pi') X$ 
  by simp
  have rew2:
     $\text{swap } ([(\text{swapas } pi \ a, \text{swapas } pi \ b)] \text{ @ } pi) (Susp \pi' X)$ 
     $= Susp ([(\text{swapas } pi \ a, \text{swapas } pi \ b)] \text{ @ } pi \text{ @ } \pi') X$ 
  by simp
  have fresh:
     $\forall c \in ds \ (pi \text{ @ } [(a,b)] \text{ @ } \pi') \ ([(\text{swapas } pi \ a, \text{swapas } pi \ b)] \text{ @ } pi \text{ @ } \pi'). \ (c, X) \in$ 
  nabla
  using swapas-append swapas-comm ds-def by simp
  from rew1 rew2 fresh
  show ?case
  using equ-susp by simp
next
  case Unit
  then show ?case by force
next
  case (Atom  $c$ )
  then show ?case
  using equ-atom swapas-comm by auto
next
  case (Paar  $t1 \ t2$ )
  then show ?case by force
next
  case (Func  $f \ t$ )
  then show ?case by force
qed

```

```

lemma l3-jud:
  assumes  $(nabla \vdash t1 \approx t2)$ 
  shows  $(nabla \vdash a \# t1) \longrightarrow (nabla \vdash a \# t2)$ 
  using assms
proof(induction rule: equ.induct)
  case (equ-abst-ab  $b \ c \ \nabla \ t2 \ t1$ )
  then show ?case
  using fresh-swap-left Fresh-elim1 fresh-abst-aa fresh-abst-ab rev-singleton-conv
  swpa.simps swapas.simps(1,2)
  by metis
next
  case (equ-abst-aa  $\nabla \ t1 \ t2 \ b$ )
  then show ?case
  using fresh-swap-left Fresh-elim1 fresh-abst-aa fresh-abst-ab
  by metis
next

```

```

    case (equ-unit nabla)
    then show ?case
      by blast
  next
    case (equ-atom b c nabla)
    then show ?case
      by simp
  next
    case (equ-susp pi1 pi2 X nabla)
    then show ?case
      using Fresh-elims(4) ds-elem-cp ds-rev fresh-susp swapas-append swapas-rev-pi-a
      by metis
  next
    case (equ-paar nabla t1 t2 s1 s2)
    then show ?case
      using Fresh-elims(5) by blast
  next
    case (equ-func nabla t1 t2 f)
    then show ?case
      using Fresh-elims(6) by blast
qed

```

```

lemma ds-empty-equiv-1:
  assumes ds pi1 pi2 = {}
  shows nabla ⊢ swap pi1 t1 ≈ t2 ⇒ nabla ⊢ swap pi2 t1 ≈ t2
  using assms
proof(induction t1 arbitrary: t2)
  case (Abst a t1')
  then obtain b t2'
    where t2: t2 = Abst b t2'
    by(auto elim: equ.cases)
  with Abst have i: nabla ⊢ Abst (swapas pi1 a) (swap pi1 t1') ≈ Abst b t2'
    using swap.simps(4) by simp
  then show ?case
  proof(cases ⟨swapas pi1 a = b⟩)
    case True
    then have nabla ⊢ swap pi1 t1' ≈ t2'
      using Abst i Equ-elims(6) by blast
    then have ii: nabla ⊢ swap pi2 t1' ≈ t2'
      using Abst by simp
    then have nabla ⊢ Abst (swapas pi2 a) (swap pi2 t1') ≈ Abst b t2'
      using True ds-swapas-eq[OF Abst(3), of a] equ-abst-aa[OF ii, of ⟨swapas pi2
a⟩]
      by simp
    then show ?thesis
      using t2 by simp
  qed

```

```

next
  case False
  then have equ-ab-pi1: nabl $\vdash$  swap pi1 t1'  $\approx$  swap [(swapas pi1 a, b)] t2'
    and fresh-ab-pi1: nabl $\vdash$  swapas pi1 a  $\#$  t2'
    using i Equ-elim $s$ (7)[of nabl $\langle$ swapas pi1 a $\rangle$   $\langle$ swap pi1 t1' $\rangle$  b t2']
    by blast+
  have equ-ab-pi2: nabl $\vdash$  swap pi2 t1'  $\approx$  swap [(swapas pi2 a, b)] t2'
    using equ-ab-pi1 ds-swapas-eq[OF Abst(3), of a]
      Abst(3) Abst(1)[of  $\langle$ swap [(swapas pi1 a, b)] t2' $\rangle$ ] by auto
  have fresh-ab-pi2: nabl $\vdash$  swapas pi2 a  $\#$  t2'
    using fresh-ab-pi1 ds-swapas-eq[OF Abst(3), of a] by simp
  with equ-ab-pi2 have nabl $\vdash$  Abst (swapas pi2 a) (swap pi2 t1')  $\approx$  Abst b t2'
    using equ-abst-ab[OF False, of nabl $\langle$ t2 $\rangle$ ] ds-swapas-eq[OF Abst(3), of a] by
simp
  then show ?thesis using t2 by simp
qed
next
  case (Susp pi X)
  then obtain pi'
    where t2: t2 = Susp pi' X
    by (auto elim: equ.cases)
  with Susp have i: nabl $\vdash$  Susp (pi1 @ pi) X  $\approx$  Susp pi' X
    by simp
  then have  $\forall c \in ds$  (pi1@pi) pi'. (c,X)  $\in$  nabl $\langle$ a $\rangle$ 
    using Equ-elim $s$ (3)[OF i] by auto
  with Susp have  $\forall c \in ds$  (pi2@pi) pi'. (c,X)  $\in$  nabl $\langle$ a $\rangle$ 
    using ds-cancel-pi-left ds-sym ds-trans
    by blast
  then have nabl $\vdash$  Susp (pi2 @ pi) X  $\approx$  Susp pi' X
    using equ-susp by simp
  then show ?case using t2 swap.simp $s$ (3) by simp
next
  case (Atom a)
  then show ?case
    using ds-swapas-eq[OF Atom(2), of a]
    by (auto elim: equ.cases)
qed (auto elim: equ.cases)

```

```

lemma ds-empty-equiv-2:
  assumes ds pi1 pi2 = {}
  shows nabl $\vdash$  t1  $\approx$  swap pi1 t2  $\implies$  nabl $\vdash$  t1  $\approx$  swap pi2 t2
using assms
proof(induction t2 arbitrary: pi1 pi2 t1)
  case (Abst b t2')
  then obtain a t1'
    where t1: t1 = Abst a t1'
    by(auto elim: equ.cases)
  with Abst have i: nabl $\vdash$  Abst a t1'  $\approx$  Abst (swapas pi1 b) (swap pi1 t2')

```

```

    using swap.simps(4) by simp
  then show ?case
proof(cases ⟨swapas pi1 b = a⟩)
  case True
  then have nabl1: nabl1 ⊢ t1' ≈ swap pi1 t2'
    using Abst i Equ-elim(6) by blast
  then have ii: nabl1 ⊢ t1' ≈ swap pi2 t2'
    using Abst by simp
  then have nabl1: nabl1 ⊢ Abst a t1' ≈ Abst (swapas pi2 b) (swap pi2 t2')
    using True ds-swap-eq[OF Abst(3), of b] equ-abst-aa[OF ii, of ⟨swapas pi2
b⟩]
    by simp
  then show ?thesis
    using t1 by simp
next
  case False
  then have equ-ab: nabl1 ⊢ t1' ≈ swap [(a, swapas pi1 b)] (swap pi1 t2')
    and fresh-ab-pi1: nabl1 ⊢ a # swap pi1 t2'
    using i Equ-elim(7) by blast+

  then have equ-ab-pi1: nabl1 ⊢ t1' ≈ swap [(a, swapas pi1 b)]@ pi1 t2'
    using swap-append[of ⟨[(a, swapas pi1 b)]⟩ pi1 t2'] by simp

  have ds-empty: ds [(a, swapas pi1 b)] @ pi1 [(a, swapas pi2 b)] @ pi2 = {}
    using Abst(3) ds-swap-eq[OF Abst(3), of b]
    ds-cancel-pi-front[of ⟨[(a, swapas pi1 b)]⟩ pi1 pi2] by simp

  hence nabl1 ⊢ t1' ≈ swap [(a, swapas pi2 b)]@ pi2 t2'
    using Abst(1)[OF equ-ab-pi1 ds-empty] by simp
  hence equ-ab-pi2: nabl1 ⊢ t1' ≈ swap [(a, swapas pi2 b)] (swap pi2 t2')
    using swap-append[of ⟨[(a, swapas pi2 b)]⟩ pi2 t2'] by simp

  have fresh-ab-pi2: nabl1 ⊢ a # swap pi2 t2'
    using ds-empty-fresh-2 Abst(3) fresh-ab-pi1 by auto
  with equ-ab-pi2 have nabl1 ⊢ Abst a t1' ≈ Abst (swapas pi2 b) (swap pi2 t2')
    using equ-abst-ab ds-swap-eq False asms Abst(3) by auto
  then show ?thesis using t1 by simp
qed
next
  case (Susp pi' X)
  then obtain pi
    where t1: t1 = Susp pi X
    by (auto elim: equ.cases)
  with Susp have i: nabl1 ⊢ Susp pi X ≈ Susp (pi1 @ pi') X
    by simp
  then have ∀ c ∈ ds pi (pi1 @ pi'). (c, X) ∈ nabl1
    using Equ-elim(3)[OF i] by auto
  with Susp have ∀ c ∈ ds pi (pi2 @ pi'). (c, X) ∈ nabl1
    using ds-cancel-pi-left ds-sym ds-trans

```

```

    by blast
  then have  $nabla \vdash \text{Susp } \pi \, X \approx \text{Susp } (\pi_2 @ \pi') \, X$ 
    using equ-susp by simp
  then show ?case using t1 swap.simps(3) by simp
next
  case (Atom a)
  then show ?case
    using ds-swaps-eq by (auto elim: equ.cases)
qed (auto elim: equ.cases)

lemma equ-equivariance:
  assumes  $nabla \vdash t_1 \approx t_2$ 
  shows  $nabla \vdash \text{swap } \pi \, t_1 \approx \text{swap } \pi \, t_2$ 
  using assms
proof(induct rule: equ.induct)
  case (equ-abst-ab a b  $nabla \vdash t_2' t_1'$ )
  have i:  $nabla \vdash \text{swaps } \pi \, a \# \text{swap } \pi \, t_2'$ 
    using fresh-swap-eqv[OF equ-abst-ab(2), of  $\pi$ ] by simp

  have  $nabla \vdash \text{swap } \pi \, t_1' \approx \text{swap } (\pi @ [(a,b)]) \, t_2'$ 
    using equ-abst-ab(4) swap-append[of  $\pi \, \langle [(a,b)] \rangle \, t_2'$ ] by simp
  then have  $nabla \vdash \text{swap } \pi \, t_1' \approx \text{swap } ([(\text{swaps } \pi \, a, \text{swaps } \pi \, b)] @ \pi) \, t_2'$ 
    using ds-empty-equiv-2[OF ds-comm[of  $\pi \, a \, b$ ]] by auto
  then have ii:  $nabla \vdash \text{swap } \pi \, t_1' \approx \text{swap } [(\text{swaps } \pi \, a, \text{swaps } \pi \, b)] \, (\text{swap } \pi \, t_2')$ 
    using ds-empty-equiv-2[OF ds-comm[of  $\pi \, a \, b$ ]]
      swap-append[of  $\langle [(\text{swaps } \pi \, a, \text{swaps } \pi \, b)] \rangle \, \pi \, t_2'$ ] by simp

  have iii:  $\text{swaps } \pi \, a \neq \text{swaps } \pi \, b$ 
    using equ-abst-ab.hyps(1) swaps-rev- $\pi$ -a by blast

  with i ii
  have  $nabla \vdash \text{Abst } (\text{swaps } \pi \, a) \, (\text{swap } \pi \, t_1') \approx \text{Abst } (\text{swaps } \pi \, b) \, (\text{swap } \pi \, t_2')$ 
    using equ.equ-abst-ab by auto
  hence  $nabla \vdash \text{swap } \pi \, (\text{Abst } a \, t_1') \approx \text{swap } \pi \, (\text{Abst } b \, t_2')$ 
    using swap.simps(4) by simp
  then show ?case by simp
next
  case (equ-abst-aa  $nabla \vdash t_1' t_2' a$ )
  then show ?case
    using equ.equ-abst-aa by simp
next
  case (equ-susp  $\pi_1 \, \pi_2 \, X \, nabla$ )
  then show ?case using ds-cancel- $\pi$ -front
    by auto
qed (auto)

lemma swap-inv-side:

```

shows $\text{nabla} \vdash \text{swap } \pi \, t1 \approx t2 = \text{nabla} \vdash t1 \approx \text{swap } (\text{rev } \pi) \, t2$
proof

{**assume** $\text{assm1}: \text{nabla} \vdash \text{swap } \pi \, t1 \approx t2$
from $\text{equ-equivariance}[OF \, \text{assm1}, \text{of } \langle \text{rev } \pi \rangle]$
have $\text{nabla} \vdash \text{swap } (\text{rev } \pi \, @ \, \pi) \, t1 \approx \text{swap } (\text{rev } \pi) \, t2$
using $\text{swap-append}[\text{of } \langle \text{rev } \pi \rangle \, \pi \, t1]$ **by** simp
then show $\text{nabla} \vdash t1 \approx \text{swap } (\text{rev } \pi) \, t2$
using $\text{ds-empty-equiv-1}[OF \, \text{ds-rev-pi-id}[\text{of } \pi], \text{of } \text{nabla} \, t1 \, \langle \text{swap } (\text{rev } \pi) \, t2 \rangle]$
 $\text{swap-id}[\text{of } t1]$ **by** simp }

{**assume** $\text{assm2}: \text{nabla} \vdash t1 \approx \text{swap } (\text{rev } \pi) \, t2$
from $\text{equ-equivariance}[OF \, \text{assm2}, \text{of } \pi]$
have $\text{nabla} \vdash \text{swap } \pi \, t1 \approx \text{swap } (\pi \, @ \, \text{rev } \pi) \, t2$
using $\text{swap-append}[\text{of } \pi \, \langle \text{rev } \pi \rangle \, t2]$ **by** simp
then show $\text{nabla} \vdash \text{swap } \pi \, t1 \approx t2$
using $\text{ds-empty-equiv-2}[OF \, \text{ds-pi-rev-id}[\text{of } \pi], \text{of } \text{nabla} \, \langle \text{swap } \pi \, t1 \rangle \, t2]$
 $\text{swap-id}[\text{of } t2]$ **by** simp }

qed

lemma equ-swap-abba :
assumes $n = \text{depth } t1$
shows $\text{nabla} \vdash \text{swap } [(a,b)] \, t1 \approx t2 \implies \text{nabla} \vdash t1 \approx \text{swap } [(b,a)] \, t2$
proof–
assume $\text{nabla} \vdash \text{swap } [(a,b)] \, t1 \approx t2$
with $\text{ds-baab}[\text{of } b \, a]$
have $\text{nabla} \vdash \text{swap } [(b,a)] \, t1 \approx t2$
using ds-empty-equiv-1 **by** blast
then show $?thesis$
using $\text{swap-inv-side}[\text{of } \text{nabla} \, \langle [(b,a)] \rangle]$ **by** simp

qed

lemma equ-equiv-pi :
assumes $\forall a \in \text{ds } \pi1 \, \pi2. \text{nabla} \vdash a \# t$
shows $\text{nabla} \vdash \text{swap } \pi1 \, t \approx \text{swap } \pi2 \, t$
using $\text{assms } \text{equ-pi-right}[\text{of } \langle \text{rev } \pi1 \, @ \, \pi2 \rangle \, \text{nabla} \, t]$
 $\text{swap-inv-side}[\text{of } \text{nabla} \, \pi1 \, t \, \langle \text{swap } \pi2 \, t \rangle]$ $\text{ds-rev swap-append}$ **by** simp

lemma equ-symm :
shows $(\text{nabla} \vdash t1 \approx t2) \implies (\text{nabla} \vdash t2 \approx t1)$
proof($\text{induction rule: equ.induct}$)
case $(\text{equ-abst-ab } a \, b \, \text{nabla} \, t2' \, t1')$
have $i: \text{nabla} \vdash b \# \text{swap } [(a, b)] \, t2'$
using $\text{fresh-swap-eqv}[\text{of } \text{nabla} \, a \, t2' \, [(a,b)]] \, \text{equ-abst-ab}(2)$ **by** auto
with $\text{equ-abst-ab}(4)$
have $b\text{-fresh}: \text{nabla} \vdash b \# t1'$
using l3-jud **by** blast
from $\text{equ-abst-ab}(4)$

```

have ii:  $\text{nabla} \vdash t2' \approx \text{swap} [(b, a)] t1'$ 
  using equ-swap-abba by simp
show ?case
  using equ.equ-abst-ab[OF equ-abst-ab(1)[symmetric] b-fresh ii] by simp
next
case (equ-abst-aa  $\text{nabla} t1' t2' a$ )
then show ?case
  using equ.equ-abst-aa[OF equ-abst-aa(2), of a] by simp
next
case (equ-unit  $\text{nabla}$ )
then show ?case by auto
next
case (equ-atom a b  $\text{nabla}$ )
then show ?case by auto
next
case (equ-susp pi1 pi2 X  $\text{nabla}$ )
then show ?case
  using ds-sym by auto
next
case (equ-paar  $\text{nabla} t1' t2' s1' s2'$ )
then show ?case by auto
next
case (equ-func  $\text{nabla} t1' t2' f$ )
then show ?case
  using equ.equ-func[OF equ-func(2), of f] by simp
qed

lemma equ-trans:
  assumes  $\text{nabla} \vdash t1 \approx t2$   $\text{nabla} \vdash t2 \approx t3$ 
  shows  $\text{nabla} \vdash t1 \approx t3$ 
  using assms
proof(induction <depth t1> arbitrary: t1 t2 t3 rule: nat-less-induct)
  case 1
  note IH = this
  have IH-usable:
     $\text{depth } t1' < \text{depth } t1 \implies \text{nabla} \vdash t1' \approx t2' \implies \text{nabla} \vdash t2' \approx t3' \implies \text{nabla} \vdash t1' \approx t3'$ 
  for t1' t2' t3' using IH by blast
  have  $\text{nabla} \vdash t1 \approx t2 \implies \text{nabla} \vdash t2 \approx t3 \implies \text{nabla} \vdash t1 \approx t3$ 
  proof-
    assume t12:  $\text{nabla} \vdash t1 \approx t2$  and t23:  $\text{nabla} \vdash t2 \approx t3$ 
    show ?thesis
    proof(cases rule: equ.cases[OF <nabla t1 t2>])
      case (1 a b  $\text{nabla} t2' t1'$ )
      from 1(2) have deptht1:  $\text{depth } t1' < \text{depth } t1$ 
      using depth.simps(4) by auto
      from t23 show ?thesis
      proof(cases rule: equ.cases)

```

```

case (equ-abst-ab b c t3' t2')
then show ?thesis
proof(cases a = c)
  case True
    have i: nabla  $\vdash$  swap [(c, b)] t2' ≈ swap ([(c, b)]@[(b, c)]]) t3'
      using equ-equivariance[OF equ-abst-ab(5), of  $\langle$ [(c, b)] $\rangle$  1(1)
        swap-append[of  $\langle$ [(c, b)] $\rangle$   $\langle$ [(b, c)] $\rangle$  t3'] by simp
    have ii: nabla  $\vdash$  swap [(c, b)] t2' ≈ t3'
      using ds-empty-equiv-2[OF ds-baab-id[OF equ-abst-ab(3)] i] by simp
    with equ-abst-ab 1 True
    have iii: nabla  $\vdash$  t1' ≈ swap [(c, b)] t2' by blast
    with ii have nabla  $\vdash$  t1' ≈ t3'
      using IH-usable[OF deptht1, of  $\langle$ swap [(c, b)] t2' $\rangle$  t3'] 1(1)
      by simp
    then show ?thesis using True equ-abst-ab(2) 1(1,2) by auto
  next
    case False

    have deptht2: depth (swap [(a, b)] t2') < depth t1
      using depth.simps(4) equ-depth[OF 1(6)] swap-depth 1(3) deptht1
      equ-abst-ab(1) by simp

    from equ-abst-ab(1,4,5) 1(1,3,5)
      have ii: nabla  $\vdash$  a  $\#$  swap [(b, c)] t3'
        using l3-jud by simp

    then have a-fresh-t3: nabla  $\vdash$  a  $\#$  t3'
      using fresh-swap-left[OF ii] False equ-abst-ab 1 by simp
    have b-fresh-t3: nabla  $\vdash$  b  $\#$  t3'
      using equ-abst-ab(4) 1(1) by simp

    from 1(1) equ-abst-ab(5)
    have nabla  $\vdash$  swap [(a, b)] t2' ≈ swap ([(a, b)]@[(b, c)]]) t3'
      using equ-equivariance[OF equ-abst-ab(5)]
        swap-append[of  $\langle$ [(a, b)] $\rangle$   $\langle$ [(b, c)] $\rangle$  t3'] by simp
    then have iii: nabla  $\vdash$  swap [(a, b)] t2' ≈ swap ([(a, b),(b, c)]]) t3'
      by simp

    have ds [(a, b), (b, c)] [(a, c)] = {a, b}
      using ds-acabbc equ-abst-ab 1 False by simp
    with a-fresh-t3 b-fresh-t3
    have iv: nabla  $\vdash$  swap [(a, b),(b, c)] t3' ≈ swap [(a, c)] t3'
      using equ-equiv-pi by simp

    with 1(1) iii have nabla  $\vdash$  swap [(a, b)] t2' ≈ swap [(a, c)] t3'
      using IH-usable[OF deptht2, of  $\langle$ swap ([(a, b),(b, c)]]) t3' $\rangle$   $\langle$ swap [(a, c)]
t3' $\rangle$ ]
      by simp

```



```

with 1(1,3,6) have  $\text{nabla} \vdash t1' \approx \text{swap} [(a,c)] t3'$ 
  using IH-usable[OF deptht1, of  $\langle \text{swap} [(a,b)] t2' \rangle \langle \text{swap} [(a,c)] t3' \rangle$ ]
  equ-abst-ab(1) by simp

with 1(1,2) equ-abst-ab(2)
show ?thesis using equ.equ-abst-ab[OF False a-fresh-t3] by simp
qed
next
case (equ-abst-aa t2' t3' b)
from this(3) 1(1)
have  $i: \text{nabla} \vdash \text{swap} [(a,b)] t2' \approx \text{swap} [(a,b)] t3'$ 
  using equ-equivariance[of  $\text{nabla} t2' t3' \langle [(a,b)] \rangle$ ] by simp

have  $\text{nabla} \vdash b \# \text{swap} [(a,b)] t2'$ 
  using 1(1,3,4,5) equ-abst-aa(1) fresh-swap-eqvt[of  $\text{nabla} a t2' \langle [(a,b)] \rangle$ ]
  by auto
then have  $ii: \text{nabla} \vdash b \# \text{swap} [(a,b)] t3'$ 
  using l3-jud[OF i, of b] by simp
have  $\text{nabla} \vdash \text{swapas} [(a,b)] b \# t3'$ 
  using fresh-swap-left[OF ii] by simp
then have a-fresh-t3:  $\text{nabla} \vdash a \# t3'$ 
  using 1(3,4) equ-abst-aa(1) by simp

have is-equ:  $\text{nabla} \vdash t1' \approx \text{swap} [(a,b)] t3'$ 
  using 1 i IH-usable[OF deptht1, of  $\langle \text{swap} [(a,b)] t2' \rangle \langle \text{swap} [(a,b)] t3' \rangle$ ]
  equ-abst-aa(1) by auto

from a-fresh-t3 is-equ show ?thesis
  using 1 equ-abst-aa(1,2) equ.equ-abst-ab by simp
qed (auto simp add: 1 t12)
next
case (2  $\text{nabla} t1' t2' a$ )
have deptht1:  $\text{depth } t1' < \text{depth } t1$ 
  using depth.simps(4) 2(2) by simp
from t23 show ?thesis
proof(cases rule: equ.cases)
  case (equ-abst-ab a c t3' t2')
  have  $\text{nabla} \vdash t1' \approx \text{swap} [(a,c)] t3'$ 
    using 2(1,2,3,4) IH-usable[OF deptht1, of  $\langle t2' \rangle \langle \text{swap} [(a,c)] t3' \rangle$ ]
    equ-abst-ab(1,4,5) by simp
  then show ?thesis
    using equ.equ-abst-ab[OF equ-abst-ab(3)]
    2(1,2,3) equ-abst-ab(1,2,4) by auto
next
  case (equ-abst-aa t2' t3' a)
  then show ?thesis
    using 1 2(1,2,3,4) by auto
qed (auto simp add: 2)
next

```

```

case (3 nablā)
  from t23 show ?thesis
  proof(cases rule: equ.cases)
    case equ-unit
    then show ?thesis using t12 by auto
  qed (auto simp add: 3)
next
case (4 a b nablā)
  from t23 show ?thesis
  proof(cases rule: equ.cases)
    case (equ-atom a b)
    then show ?thesis
      using t12 by auto
  qed (auto simp add: 4)
next
case (5 pi1 pi2 X nablā)
  from t23 show ?thesis
  proof(cases rule: equ.cases)
    case (equ-susp pi1 pi2 X)
    then show ?thesis
      using ds-trans 5 by blast
  qed (auto simp add: 5)
next
case (6 nablā t1' t2' s1' s2')
  from t23 show ?thesis
  proof(cases rule: equ.cases)
    case (equ-paar t2' t3' s2' s3')
    have deptht1: depth t1' < depth t1
      using 6(2) by simp
    have depths1: depth s1' < depth t1
      using 6(2) by simp
    have a: nablā ⊢ t1' ≈ t2'
      using 6(3,4) equ-paar by auto
    have b: nablā ⊢ s1' ≈ s2'
      using 6 equ-paar by auto
    from a have t1-equal-t3: nablā ⊢ t1' ≈ t3'
      using IH-usable[of t1' t2' t3] equ-paar(3) 6(1) deptht1
      by simp
    from b have s1-equal-s3: nablā ⊢ s1' ≈ s3'
      using IH-usable[of s1' s2' s3] equ-paar(4) 6(1) depths1
      by simp
    from t1-equal-t3 s1-equal-s3 show ?thesis
      using equ.equ-paar[of nablā t1' t3' s1' s3] 6(1,2) equ-paar(2)
      by simp
  qed (auto simp add: 6)
next
case (7 nablā t1 t2 F)
  from t23 show ?thesis
  proof(cases rule: equ.cases)

```

```

      case (equ-func t1 t2 F)
      then show ?thesis
        using 1  $\gamma(1,2,3,4)$  by auto
      qed (auto simp add:  $\gamma$ )
    qed
  qed
  then show ?case
    using IH(2,3) by blast
  qed

```

```

lemma pi-right-eq-help:
  assumes (n = depth t)
  shows  $\text{nabla} \vdash t \approx \text{swap } \pi \ t \implies \forall a \in \text{ds} \ [] \ \pi. \text{nabla} \vdash a \# t$ 
  using assms
proof(induction n arbitrary: t  $\pi$  rule: nat-less-induct)
  case (1 n)
  note IH = this
  then show ?case
  proof(cases rule: equ.cases[OF  $\langle \text{nabla} \vdash t \approx \text{swap } \pi \ t \rangle$ ])
    case (1 b c  $\text{nabla} \ t2 \ t1$ )
    have deptht1:  $\text{depth } t1 < n$ 
    using 1 depth.simps(4) IH by simp
    have swap- $\pi$ - $t1$ -is- $t2$ :  $\text{swap } \pi \ t1 = t2$ 
    using 1(2,3) by auto
    have swap- $\pi$ - $b$ -is- $c$ :  $\text{swapas } \pi \ b = c$ 
    using 1(2,3) by auto
    with swap- $\pi$ - $t1$ -is- $t2$  have  $\text{nabla} \vdash t1 \approx \text{swap } [(b, \text{swapas } \pi \ b)] (\text{swap } \pi$ 
  t1)
    using 1(1,6) by simp
  then have  $\text{nabla} \vdash t1 \approx \text{swap } [(b, \text{swapas } \pi \ b)] @ \pi) \ t1$ 
    using swap-append[of  $\langle [(b, \text{swapas } \pi \ b)] \rangle \ \pi \ t1$ ] by simp
  with deptht1 have  $\forall a \in \text{ds} \ [] \ ((b, \text{swapas } \pi \ b) \# \pi). \text{nabla} \vdash a \# t1$ 
    using 1.IH 1 by auto
  then have ds-minus-bc:  $\forall a \in \text{ds} \ [] \ \pi - \{b, c\}. \text{nabla} \vdash a \# t1$ 
    using ds-equality swap- $\pi$ - $b$ -is- $c$  by auto
  have c-fresh-t1:  $\text{nabla} \vdash c \# t1$ 
    using 1 equ-symm l3-jud fresh-swap-eqv Equ-elim equ-abst-ab by meson
  with ds-minus-bc have  $\forall a \in \text{ds} \ [] \ \pi - \{b\}. \text{nabla} \vdash a \# t1$ 
    by auto
  then have ds-minus-b:  $\forall a \in \text{ds} \ [] \ \pi - \{b\}. \text{nabla} \vdash a \# \text{Abst } b \ t1$ 
    using fresh-abst-ab by blast
  have b-fresh-abst-t1:  $\text{nabla} \vdash b \# \text{Abst } b \ t1$ 
    using fresh-abst-aa by simp
  have  $\forall a \in \text{ds} \ [] \ \pi. \text{nabla} \vdash a \# \text{Abst } b \ t1$ 
  proof(rule, goal-cases)
    case (1 a)
    then show ?case

```

```

      using b-fresh-abst-t1 ds-minus-b
      by (cases a=b, auto)
    qed
  with 1 show ?thesis
    by auto
next
case (2 nabla t1 t2 b)
have deptht1: depth t1 < n
  using 2 depth.simps(4) IH by simp
have swap-pi-t1-is-t2: swap pi t1 = t2
  using 2(2,3) by auto
from swap-pi-t1-is-t2 deptht1
have  $\forall a \in ds \ [] \text{ pi. nabla } \vdash a \# t1$ 
  using 1.IH 2(1,4) by auto
then show ?thesis
  using 2(1,2,3) elem-ds by fastforce
next
case (3 nabla)
then show ?thesis
  by blast
next
case (4 a b nabla)
then show ?thesis
  using elem-ds by force
next
case (5 pi1 pi2 X nabla)
have swap pi t = Susp (pi@pi1) X
  using 5(2) by auto
also have ... = Susp pi2 X
  using 5(3) calculation by simp
then have pi@pi1 = pi2 by simp
hence  $\forall c \in ds \text{ pi1 } (pi@pi1). (c, X) \in nabla$ 
  using 5(4) by simp
then show ?thesis
  using 5(1,2) fresh-susp ds-cancel-pi-right[of - - [] -]
  by simp
next
case (6 nabla t1 t2 s1 s2)
have deptht1: depth t1 < n
  using 6 depth.simps(5) IH
  by simp
have depths1: depth s1 < n
  using 6 depth.simps(5) IH
  by simp
from deptht1 depths1 show ?thesis
  using 1.IH 6 by auto
next
case (7 nabla t1 t2 f)
have deptht1: depth t1 < n

```

```

    using 7(2) depth.simps(5) IH
    by auto
  have  $\text{nabla} \vdash t1 \approx \text{swap } \pi \ t1$ 
    using 7 by simp
  then have  $\forall a \in ds \ [] \ \pi. \ \text{nabla} \vdash a \# t1$ 
    using 7(1) IH depth1 by simp
  then show ?thesis
    using 7(1,2) fresh-func[of  $\text{nabla} - t1 \ f$ ]
    by simp
qed
qed

end
theory Equ

imports PreEqu

begin

lemma equ-refl:
   $\text{nabla} \vdash t \approx t$ 
  by(induct t, auto simp add: ds-def)

lemma equ-dec-pi:
   $\text{nabla} \vdash \text{swap } \pi \ t1 \approx \text{swap } \pi \ t2 \implies \text{nabla} \vdash t1 \approx t2$ 
proof-
  have  $i: \text{nabla} \vdash \text{swap } (\text{rev } \pi) (\text{swap } \pi \ t1) \approx t1$ 
     $\text{nabla} \vdash \text{swap } (\text{rev } \pi) (\text{swap } \pi \ t2) \approx t2$ 
    using rev-pi-pi-equ by auto
  assume  $\text{nabla} \vdash \text{swap } \pi \ t1 \approx \text{swap } \pi \ t2$ 
  then have  $\text{nabla} \vdash \text{swap } (\text{rev } \pi) (\text{swap } \pi \ t1) \approx \text{swap } (\text{rev } \pi) (\text{swap } \pi \ t2)$ 
    using equ-equivariance by simp
  then show ?thesis using i equ-symm equ-trans by meson
qed

lemma equ-involutive-left:
   $\text{nabla} \vdash \text{swap } (\text{rev } \pi) (\text{swap } \pi \ t1) \approx t2 = \text{nabla} \vdash t1 \approx t2$ 
proof(auto)
  have  $i: \text{nabla} \vdash t1 \approx \text{swap } (\text{rev } \pi) (\text{swap } \pi \ t1)$ 
    using rev-pi-pi-equ equ-symm by blast
  show  $\text{nabla} \vdash \text{swap } (\text{rev } \pi) (\text{swap } \pi \ t1) \approx t2 \implies \text{nabla} \vdash t1 \approx t2$ 
    using i equ-trans by blast
  show  $\text{nabla} \vdash t1 \approx t2 \implies \text{nabla} \vdash \text{swap } (\text{rev } \pi) (\text{swap } \pi \ t1) \approx t2$ 
    using i equ-trans equ-symm by blast
qed

```

lemma *equ-pi-to-left*:

$nabla \vdash \text{swap} (\text{rev } \pi) \ t1 \approx t2 = \nabla \vdash t1 \approx \text{swap } \pi \ t2$

proof

```
{assume i:  $\nabla \vdash \text{swap} (\text{rev } \pi) \ t1 \approx t2$ 
have  $\nabla \vdash \text{swap } \pi (\text{swap} (\text{rev } \pi) \ t1) \approx \text{swap } \pi \ t2$ 
  using equ-equivariance[OF i, of  $\pi$ ] by simp
then show  $\nabla \vdash t1 \approx \text{swap } \pi \ t2$ 
  using equ-involutive-left[of  $\nabla \vdash \langle \text{rev } \pi \rangle \ t1 \ \langle \text{swap } \pi \ t2 \rangle$ ] rev-rev-ident[of  $\pi$ ]
  by simp}
```

```
{assume i:  $\nabla \vdash t1 \approx \text{swap } \pi \ t2$ 
have ii:  $\nabla \vdash \text{swap} (\text{rev } \pi) \ t1 \approx \text{swap} (\text{rev } \pi) (\text{swap } \pi \ t2)$ 
  using equ-equivariance[OF i, of  $\langle \text{rev } \pi \rangle$ ] by simp
then have iii:  $\nabla \vdash \text{swap} (\text{rev } \pi) (\text{swap } \pi \ t2) \approx \text{swap} (\text{rev } \pi) \ t1$ 
  using equ-symm[OF ii] by simp
then have iv:  $\nabla \vdash t2 \approx \text{swap} (\text{rev } \pi) \ t1$ 
  using equ-involutive-left[of  $\nabla \vdash \pi \ t2 \ \langle \text{swap} (\text{rev } \pi) \ t1 \rangle$ ] by simp
then show  $\nabla \vdash \text{swap} (\text{rev } \pi) \ t1 \approx t2$ 
  using equ-symm[OF iv] by simp}
```

qed

lemma *equ-pi-to-right*:

$\nabla \vdash t1 \approx \text{swap} (\text{rev } \pi) \ t2 = \nabla \vdash \text{swap } \pi \ t1 \approx t2$

proof

```
{assume i:  $\nabla \vdash t1 \approx \text{swap} (\text{rev } \pi) \ t2$ 
  then show  $\nabla \vdash \text{swap } \pi \ t1 \approx t2$ 
    using equ-involutive-left equ-dec-pi by blast}
{assume ii:  $\nabla \vdash \text{swap } \pi \ t1 \approx t2$ 
  then show  $\nabla \vdash t1 \approx \text{swap} (\text{rev } \pi) \ t2$ 
    using equ-involutive-left equ-equivariance by blast}
```

qed

lemma *equ-involutive-right*:

$\nabla \vdash t1 \approx \text{swap} (\text{rev } \pi) (\text{swap } \pi \ t2) = \nabla \vdash t1 \approx t2$

```
using equ-dec-pi[of  $\nabla \vdash \pi \ t1 \ t2$ ] equ-equivariance[of  $\nabla \vdash t1 \ t2 \ \pi$ ]
  equ-pi-to-right[of  $\nabla \vdash \pi \ t1 \ \langle \text{swap } \pi \ t2 \rangle$ ]
by auto
```

lemma *equ-pi1-pi2-add*:

$(\forall a \in ds \ \pi1 \ \pi2. \nabla \vdash a \# t) \implies (\nabla \vdash \text{swap } \pi1 \ t \approx \text{swap } \pi2 \ t)$

proof –

```
assume  $(\forall a \in ds \ \pi1 \ \pi2. \nabla \vdash a \# t)$ 
hence  $\nabla \vdash t \approx \text{swap} (\text{rev } \pi1 \ @ \ \pi2) \ t$ 
  using ds-rev equ-pi-right by simp
```

hence $nabla \vdash t \approx \text{swap} (\text{rev } \pi 1) (\text{swap } \pi 2 \ t)$
 using *swap-append* **by** *auto*
 then show $nabla \vdash \text{swap } \pi 1 \ t \approx \text{swap } \pi 2 \ t$
 using *equ-pi-to-right* **by** *simp*
qed

lemma *pi-right-equ*: $(nabla \vdash t \approx \text{swap } \pi \ t) \implies (\forall a \in ds \ [] \ \pi. \nabla \vdash a \# t)$
 using *pi-right-equ-help* **by** *blast*

lemma *equ-pi1-pi2-dec*:
 $(nabla \vdash \text{swap } \pi 1 \ t \approx \text{swap } \pi 2 \ t) \implies (\forall a \in ds \ \pi 1 \ \pi 2. \nabla \vdash a \# t)$
proof –
 assume *assm*: $nabla \vdash \text{swap } \pi 1 \ t \approx \text{swap } \pi 2 \ t$
 then have $nabla \vdash t \approx \text{swap} ((\text{rev } \pi 1) @ \pi 2) \ t$
 using *equ-pi-to-right* *swap-append* **by** *simp*
 then show $\forall a \in ds \ \pi 1 \ \pi 2. \nabla \vdash a \# t$
 using *pi-right-equ* *ds-rev[of pi1 pi2]* **by** *simp*
qed

lemma *equ-weak*:
 $nabla 1 \vdash t 1 \approx t 2 \implies (nabla 1 \cup nabla 2) \vdash t 1 \approx t 2$
by (*erule equ.induct*, *auto simp add: fresh-weak*)

lemma *psub-trm-not-equ*:
 $\forall t 2 \in \text{psub-trms } t 1. (\neg (\exists \pi. (\nabla \vdash t 1 \approx \text{swap } \pi \ t 2)))$
proof
 fix *t2*
 assume *i*: $t 2 \in \text{psub-trms } t 1$
 show $\neg (\exists \pi. \nabla \vdash t 1 \approx \text{swap } \pi \ t 2)$
proof
 assume $\exists \pi. \nabla \vdash t 1 \approx \text{swap } \pi \ t 2$
 then obtain *pi* where *H*:
 $\nabla \vdash t 1 \approx \text{swap } \pi \ t 2$ **by** *blast*

 from *equ-depth[OF H]*
 have $\text{depth } t 1 = \text{depth} (\text{swap } \pi \ t 2)$
by *simp*
 hence $\text{depth } t 1 = \text{depth } t 2$
by *simp*
 moreover have $\text{depth } t 2 < \text{depth } t 1$
 using *i* *depth-psub-trms*

```

    by simp

    ultimately show False by auto
  qed
qed

end
theory Substs

imports Equ

begin

type-synonym substs = (string × trm) list

fun look-up :: string ⇒ substs ⇒ trm where
  look-up X [] = Susp [] X |
  look-up X (x#xs) = (if (X = fst x) then (snd x) else look-up X xs)

fun subst :: substs ⇒ trm ⇒ trm where
  subst-unit: subst s (Unit) = Unit |
  subst-susp: subst s (Susp [] X) = swap pi (look-up X s) |
  subst-atom: subst s (Atom a) = Atom a |
  subst-abst: subst s (Abst a t) = Abst a (subst s t) |
  subst-paar: subst s (Paar t1 t2) = Paar (subst s t1) (subst s t2) |
  subst-func: subst s (Func F t) = Func F (subst s t)

declare subst-susp [simp del]

fun alist-rec :: substs ⇒ substs ⇒ (string⇒trm⇒substs⇒substs⇒substs) ⇒ substs
where
  alist-rec [] c d = c |
  alist-rec (p # al) c d = d (fst p) (snd p) al (alist-rec al c d)

definition comp :: substs ⇒ substs ⇒ substs (infixl ‹·› 81) where
  s1 · s2 ≡ alist-rec s2 s1 (λ x y xs g. (x,subst s1 y) # g)

definition domn :: (trm ⇒ trm) ⇒ string set where
  domn s ≡ {X. (s (Susp [] X)) ≠ (Susp [] X)}

definition ext-subst :: fresh-envs ⇒ (trm ⇒ trm) ⇒ fresh-envs ⇒ bool ( - ‹|= - -

```


$[80,80,80] \ 80)$ **where**
 $nabla1 \models s \ nabla2 \equiv (\forall (a,X) \in nabla2. \ nabla1 \vdash a^\# s \ (Susp \ [] \ X))$

definition *subst-equ* :: *fresh-envs* $\Rightarrow$ (*trm* $\Rightarrow$ *trm*) $\Rightarrow$ (*trm* $\Rightarrow$ *trm*) $\Rightarrow$ *bool* ($- \models - \approx$ $- [80,80,80] \ 80)$

where
 $nabla \models s1 \approx s2 \equiv \forall X \in (\text{domn } s1 \cup \text{domn } s2). \ (nabla \vdash s1 \ (Susp \ [] \ X) \approx s2 \ (Susp \ [] \ X))$

lemma *subst-equ-symm*:
assumes $nabla \models s1 \approx s2$
shows $nabla \models s2 \approx s1$
using *assms equ-symm subst-equ-def*[*of* $nabla \ s1 \ s2$]
subst-equ-def[*of* $nabla \ s2 \ s1$] *Un-commute*[*of* $\langle \text{domn } s1 \rangle \ \langle \text{domn } s2 \rangle$] **by** *blast*

lemma *subst-equ-refl*:
shows $nabla \models s \approx s$
using *equ-refl subst-equ-def Un-absorb* **by** *simp*

lemma *not-in-domn*: $X \notin (\text{domn } s) \Longrightarrow (s \ (Susp \ [] \ X)) = (Susp \ [] \ X)$
using *domn-def* **by** *simp*

lemma *subst-swap-comm*: $\text{subst } s \ (\text{swap } pi \ t) = \text{swap } pi \ (\text{subst } s \ t)$
using *swap-append subst-susp* **by** (*induct t*) *auto*

lemma *subst-not-occurs*:
 $\neg(\text{occurs } X \ t) \Longrightarrow \text{subst } [(X,t2)] \ t = t$
proof–
have $i: \neg(\text{occurs } X \ t) \longrightarrow \text{subst } [(X,t2)] \ t = t$
using *subst-susp* **by** (*induct t*) *auto*
assume $\neg(\text{occurs } X \ t)$
with i **show** *?thesis* **by** *simp*
qed

lemma *id-subst* [*simp*]: $\text{subst } [] \ t = t$
using *subst-susp* **by** (*induct t*) *auto*

lemma *subst-comp-id* [*simp*]: $\text{subst } (s \cdot []) = \text{subst } s$
using *comp-def* **by** *simp*

lemma *id-comp-subst* [*simp*]: $\text{subst } ([] \cdot s) = \text{subst } s$
proof(*rule ext*)
fix t
show $\text{subst } ([] \cdot s) \ t = \text{subst } s \ t$
proof(*induction t arbitrary: s*)
case (*Susp pi X*)

```

then show ?case using comp-def subst-susp
proof (induction s)
  case Nil
  then show ?case
    using comp-def subst-susp by simp
next
  case (Cons a s)
  then show ?case
    using comp-def subst-susp by simp
qed
qed (simp-all)
qed

```

```

lemma subst-comp-expand: subst (s1 · s2) t = subst s1 (subst s2 t)
proof(induct t)
  case (Susp x1 x2)
  then show ?case
    proof(induct s2)
      case Nil
      then show ?case using comp-def subst-susp subst-swap-comm by simp
    next
      case (Cons a s2)
      then show ?case using comp-def subst-susp subst-swap-comm by simp
    qed
  qed (simp-all)

```

```

lemma subst-assoc: subst (s1 · (s2 · s3)) = subst ((s1 · s2) · s3)
proof(rule ext)
  fix t
  show subst (s1 · (s2 · s3)) t = subst (s1 · s2 · s3) t
  proof(induct t)
    case (Susp pi X)
    then show ?case using subst-comp-expand by simp
  qed (simp-all)
qed

```

```

lemma fresh-subst:
  assumes nabl1 ⊢ a # t
    and nabl2 ⊨ (subst s) nabl1
  shows nabl2 ⊢ a # subst s t
  using assms
proof(induction rule: fresh.induct)
  case (fresh-susp pi a X nabl)
  then show ?case
    using ext-subst-def subst-susp fresh-swap-right

```

by auto
qed (auto)

lemma equ-subst:
assumes $nabla1 \vdash t1 \approx t2$
and $nabla2 \models (subst\ s)\ nabla1$
shows $nabla2 \vdash (subst\ s\ t1) \approx (subst\ s\ t2)$
using *assms*
proof(induction rule: equ.induct)
case (equ-abst-ab a b $nabla\ t2\ t1$)
then show ?case
using subst-swap-comm fresh-subst equ.equ-abst-ab by simp
next
case (equ-susp $\pi1\ \pi2\ X\ nabla$)
then show ?case
using subst-susp equ- $\pi1$ - $\pi2$ -add ext-subst-def subst-susp
by fastforce
qed (auto)

lemma unif-1:
assumes $nabla \vdash subst\ s\ (Susp\ \pi\ X) \approx subst\ s\ t$
shows $nabla \models subst\ (s \cdot [(X, swap\ (rev\ \pi)\ t)]) \approx subst\ s$
using *assms*
apply(simp only: subst-equ-def)
proof
fix Xa
assume *hyps*: $nabla \vdash subst\ s\ (Susp\ \pi\ X) \approx subst\ s\ t$
 $Xa \in domn\ (subst\ (s \cdot [(X, swap\ (rev\ \pi)\ t)])) \cup domn\ (subst\ s)$
show $nabla \vdash subst\ (s \cdot [(X, swap\ (rev\ \pi)\ t)])\ (Susp\ []\ Xa) \approx subst\ s\ (Susp\ []\ Xa)$
proof-
have one:
 $nabla \vdash subst\ (s \cdot [(X, swap\ (rev\ \pi)\ t)])\ (Susp\ \pi\ Xa) \approx$
 $subst\ s\ (subst\ [(X, swap\ (rev\ \pi)\ t)]\ (Susp\ \pi\ Xa))$
using subst-comp-expand equ-refl by simp
have two:
 $nabla \vdash subst\ s\ (subst\ [(X, swap\ (rev\ \pi)\ t)]\ (Susp\ \pi\ Xa)) \approx$
 $subst\ s\ (swap\ \pi\ (look-up\ Xa\ [(X, swap\ (rev\ \pi)\ t)]))$
using equ-refl subst-susp[of $\langle [(X, swap\ (rev\ \pi)\ t)] \rangle\ \pi\ Xa]$
by simp
have $nabla \vdash subst\ (s \cdot [(X, swap\ (rev\ \pi)\ t)])\ (Susp\ \pi\ Xa) \approx$
 $subst\ s\ (swap\ \pi\ (look-up\ Xa\ [(X, swap\ (rev\ \pi)\ t)]))$
using equ-trans[OF one two] by simp
hence $nabla \vdash swap\ \pi\ (look-up\ Xa\ (s \cdot [(X, swap\ (rev\ \pi)\ t)])) \approx$
 $subst\ s\ (swap\ \pi\ (look-up\ Xa\ [(X, swap\ (rev\ \pi)\ t)]))$
using subst-susp[of $\langle (s \cdot [(X, swap\ (rev\ \pi)\ t)]) \rangle\]$ by simp
hence swap- π -outside: $nabla \vdash swap\ \pi\ (look-up\ Xa\ (s \cdot [(X, swap\ (rev\ \pi)\ t)])) \approx$

```

swap pi (subst s (look-up Xa [(X, swap (rev pi) t)]))
  using subst-swap-comm by simp
hence nablA ⊢ (look-up Xa (s · [(X, swap (rev pi) t)])) ≈
(subst s (look-up Xa [(X, swap (rev pi) t)]))
  using equ-dec-pi[OF swap-pi-outside] by simp
hence three:
nablA ⊢ subst (s · [(X, swap (rev pi) t)]) (Susp [] Xa) ≈
(subst s (look-up Xa [(X, swap (rev pi) t)]))
  using subst-susp by simp
then show ?thesis
proof(cases Xa = X)
  case True
  hence subst s (look-up Xa [(X, swap (rev pi) t)]) = subst s (swap (rev pi) t)
    by simp
  with three have
    nablA ⊢ subst (s · [(X, swap (rev pi) t)]) (Susp [] Xa) ≈ swap (rev pi)
(subst s t)
    using subst-swap-comm by auto
  hence i: nablA ⊢ swap pi (subst (s · [(X, swap (rev pi) t)]) (Susp [] Xa))
    ≈ subst s t using swap-inv-side by simp
  hence nablA ⊢ swap pi (subst (s · [(X, swap (rev pi) t)]) (Susp [] Xa))
    ≈ subst s (Susp pi Xa)
    using True equ-trans[OF i equ-symm[OF assms(1)]] by simp
  hence nablA ⊢ swap pi (subst (s · [(X, swap (rev pi) t)]) (Susp [] Xa))
    ≈ swap pi (look-up Xa s)
    using subst-susp by simp
  then show nablA ⊢ (subst (s · [(X, swap (rev pi) t)]) (Susp [] Xa))
    ≈ subst s (Susp [] Xa)
    using equ-dec-pi subst-susp by simp
next
  case False
  hence subst s (look-up Xa [(X, swap (rev pi) t)]) = subst s (Susp [] Xa)
    by simp
  with three show ?thesis by auto
qed
qed
qed

```

```

lemma subst-equ-a:
  assumes nablA ⊢ (subst s1) ≈ (subst s2)
  and nablA ⊢ (subst s2 t1) ≈ t2
  shows nablA ⊢ (subst s1 t1) ≈ t2
  using assms
proof(induct t1 arbitrary: t2)
  case (Abst a t1')
  then obtain t2' b
    where t2: t2 = Abst b t2'
    by(auto elim: equ.cases)
  with Abst(3) have

```

```

  i: nablā ⊢ Abst a (subst s2 t1') ≈ Abst b t2'
  using subst-abst by simp
then show ?case
proof(cases a=b)
  case True
  hence nablā ⊢ (subst s2 t1') ≈ t2'
    using Equ-elim(7)[OF i] by auto
  with Abst(1)[OF Abst(2), of t2']
  have nablā ⊢ subst s1 t1' ≈ t2'
    by simp
  hence nablā ⊢ Abst a (subst s1 t1') ≈ Abst b t2'
    using True equ-abst-aa by simp
  with t2 show ?thesis
    using subst-abst[of s1 a t1'] by simp
next
  case False
  hence nablā ⊢ (subst s2 t1') ≈ swap [(a,b)] t2'
    and fresh: nablā ⊢ a # t2'
    using Equ-elim(7)[OF i] by auto
  with Abst(1)[OF Abst(2), of ⟨swap [(a,b)] t2'⟩]
  have nablā ⊢ subst s1 t1' ≈ swap [(a,b)] t2'
    by simp
  hence nablā ⊢ Abst a (subst s1 t1') ≈ Abst b t2'
    using equ-abst-ab[OF False fresh] by simp
  with t2 show ?thesis
    using subst-abst[of s1 a t1'] by simp
qed
next
  case (Susp pi X)
  hence nablā ⊢ swap pi (look-up X s2) ≈ t2
    using subst-susp by simp
  hence nablā ⊢ (look-up X s2) ≈ swap (rev pi) t2
    using subst-susp swap-inv-side by simp
  hence i: nablā ⊢ subst s2 (Susp [] X) ≈ swap (rev pi) t2
    using subst-susp[of s2 ⟨[]⟩] by simp

  with Susp(1) subst-equ-def[of nablā ⟨subst s1⟩ ⟨subst s2⟩]
  have nablā ⊢ subst s1 (Susp [] X) ≈ subst s2 (Susp [] X)
    using UnCI equ-refl not-in-dmn by metis

  with i have nablā ⊢ subst s1 (Susp [] X) ≈ swap (rev pi) t2
    using equ-trans by blast
  hence nablā ⊢ (look-up X s1) ≈ swap (rev pi) t2
    using subst-susp by simp
  hence nablā ⊢ swap pi (look-up X s1) ≈ t2
    using swap-inv-side by simp
  then show ?case
    using subst-susp by simp
next

```

```

case (Paar t11 t12)
then obtain t21 t22
  where t2: t2 = Paar t21 t22
  by(auto elim: equ.cases)
with Paar(4)
have paar-i: nabra ⊢ subst s2 t11 ≈ t21
  and paar-ii: nabra ⊢ subst s2 t12 ≈ t22
  by (auto elim: equ.cases)
from t2 show ?case
  using equ-paar[OF Paar(1)[OF Paar(3) paar-i] Paar(2)[OF Paar(3) paar-ii]]
  subst-paar by simp
next
case (Func f t1')
then obtain t2'
  where t2: t2 = Func f t2'
  by(auto elim: equ.cases)
with Func(3)
have func: nabra ⊢ subst s2 t1' ≈ t2'
  by (auto elim: equ.cases)
from t2 show ?case
  using equ-func[OF Func(1)[OF Func(2) func], of f] by simp
qed (simp-all)

```

```

lemma unif-2a:
  assumes nabra ⊢ subst s1 ≈ subst s2
  and (nabra ⊢ subst s2 t1 ≈ subst s2 t2)
shows (nabra ⊢ subst s1 t1 ≈ subst s1 t2)
proof-
  have i: (nabra ⊢ subst s1 t1 ≈ subst s2 t2)
  using subst-equ-a[OF assms(1,2)] by simp
  show (nabra ⊢ subst s1 t1 ≈ subst s1 t2)
  using subst-equ-a[OF assms(1) equ-symm[OF i]] equ-symm by auto
qed

```

```

lemma unif-2b:
  assumes nabra ⊢ subst s1 ≈ subst s2
  and nabra ⊢ a ≠ subst s2 t
shows nabra ⊢ a ≠ subst s1 t
  using assms
proof(induct t)
case (Abst b t')
then show ?case
proof(cases a=b)
case True
  then show ?thesis
  using fresh-abst-aa by simp
next

```

```

    case False
  with Abst show ?thesis
    using Fresh-elim(1) Substs.subst-abst fresh-abst-ab
    by metis
qed
next
  case (Susp pi X)
  then show ?case
    using equ-refl equ-symm l3-jud subst-equ-a by meson
next
  case (Paar t1 t2)
  then show ?case
    using Fresh-elim(5) subst-paar fresh-paar
    by metis
next
  case (Func f t')
  then show ?case
    using Fresh-elim(6) subst-func fresh-func
    by metis
qed (simp-all)

```

lemma *subst-equ-to-trm*: $\text{nabla} \models \text{subst } s1 \approx \text{subst } s2 \implies \text{nabla} \vdash \text{subst } s1 \ t \approx \text{subst } s2 \ t$
 using equ-refl subst-equ-a by simp

lemma *subst-cancel-right*:
 $\llbracket \text{nabla} \models (\text{subst } s1) \approx (\text{subst } s2) \rrbracket \implies \text{nabla} \models (\text{subst } (s1 \cdot s)) \approx (\text{subst } (s2 \cdot s))$
 using subst-comp-expand subst-equ-def subst-equ-to-trm
 by simp

lemma *subst-trans*: $\llbracket \text{nabla} \models \text{subst } s1 \approx \text{subst } s2; \text{nabla} \models \text{subst } s2 \approx \text{subst } s3 \rrbracket \implies \text{nabla} \models \text{subst } s1 \approx \text{subst } s3$
 using equ-trans subst-equ-def subst-equ-to-trm by meson

lemma *occurs-sub-trm-equ*:
 $\text{occurs } X \ t1 \implies (\exists \ t2 \in \text{sub-trms } (\text{subst } s \ t1). (\exists \ pi. (\text{nabla} \vdash \text{subst } s \ (\text{Susp } pi1 \ X) \approx (\text{swap } pi \ t2))))$
proof (induct t1)
 case (Susp pi' X)
 then show ?case
 using equ-pi-to-left equ-involutive-left
 equ-refl subst-susp swap-append t-sub-trms-t
 occurs.simps(3) by metis
next
 case (Paar t11 t12)

```

then show ?case
  using Substs.subst-paar Un-iff occurs.simps(5) psub-sub-trms
    psub-trms.simps(4) by (metis)
qed(auto)

lemma ext-subst-strong:
  assumes nabla1  $\models$  (subst s) (nabla2  $\cup$  nabla3)
  shows nabla1  $\models$  (subst s) (nabla2) and nabla1  $\models$  (subst s) (nabla3)
  using assms
  unfolding ext-subst-def by auto

lemma ext-subst-id:
  shows nabla  $\models$  (subst []) nabla
  unfolding ext-subst-def id-subst by auto

end
theory Mgu

imports Substs

begin

syntax equ-prob :: trm  $\Rightarrow$  trm  $\Rightarrow$  (trm  $\times$  trm) (infix  $\approx$ ? 81)
  fresh-prob :: string  $\Rightarrow$  trm  $\Rightarrow$  (string  $\times$  trm) (infix  $\#$ ? 81)
translations
  t1  $\approx$ ? t2  $\rightarrow$  (t1, t2)
  a  $\#$ ? t  $\rightarrow$  (a, t)

type-synonym problem-type = ((trm  $\times$  trm) list)  $\times$  ((string  $\times$  trm) list)

type-synonym unifier-type = fresh-envs  $\times$  substs

definition U:: problem-type  $\Rightarrow$  (unifier-type set)
  where all-solutions-def:
    
$$U\ P \equiv \{(nabla, s). \\
      (\forall (t1, t2) \in set\ (fst\ P). \ nabla \vdash subst\ s\ t1 \approx subst\ s\ t2) \wedge \\
      (\forall (a, t) \in set\ (snd\ P). \ nabla \vdash a \# subst\ s\ t)\}$$


type-synonym eprobs = ((trm  $\times$  trm) list)
type-synonym fprobs = ((string  $\times$  trm) list)

```


type-synonym $probs = eprobs \times fprobs$

fun $vars-fprobs :: ((string \times trm) list) \Rightarrow (string set)$
where
 $vars-fprobs [] = \{\}$ |
 $vars-fprobs (x\#xs) = (vars-trm (snd x)) \cup (vars-fprobs xs)$

fun $vars-eprobs :: ((trm \times trm) list) \Rightarrow (string set)$
where
 $vars-eprobs [] = \{\}$ |
 $vars-eprobs (x\#xs) = (vars-trm (snd x)) \cup (vars-trm (fst x)) \cup (vars-eprobs xs)$

definition $vars-probs :: problem-type \Rightarrow nat$
where $vars-probs P \equiv card((vars-fprobs (snd P)) \cup (vars-eprobs (fst P)))$

definition $mgu :: problem-type \Rightarrow unifier-type \Rightarrow bool$
where $mgu P unif \equiv$
 $\forall (nabla, s1) \in U P. (\exists s2. (nabla \models (subst s2) (fst unif)) \wedge$
 $(nabla \models subst (s2 \cdot (snd unif)) \approx subst s1))$

definition $idem :: unifier-type \Rightarrow bool$
where $idem unif \equiv (fst unif) \models subst ((snd unif) \cdot (snd unif)) \approx subst (snd unif)$

definition $apply-subst-eprobs :: substs \Rightarrow eprobs \Rightarrow eprobs$
where $apply-subst-eprobs s P \equiv map (\lambda(t1, t2). (subst s t1 \approx? subst s t2)) P$

definition $apply-subst-fprobs :: substs \Rightarrow fprobs \Rightarrow fprobs$
where $apply-subst-fprobs s P \equiv map (\lambda(a, t). (a \#? subst s t)) P$

definition $apply-subst :: substs \Rightarrow problem-type \Rightarrow problem-type$
where $apply-subst s P \equiv (map (\lambda(t1, t2). (subst s t1 \approx? subst s t2)) (fst P),$
 $map (\lambda(a, t). (a \#? (subst s t))) (snd P))$

inductive $s-red :: problem-type \Rightarrow substs \Rightarrow problem-type \Rightarrow bool \quad (- \vdash - \rightsquigarrow -)$

[80,80,80] 80)

where

$unit-sred[intro!]: ((Unit \approx^? Unit) \# xs, ys) \vdash [] \rightsquigarrow (xs, ys) \mid$
 $paar-sred[intro!]: ((Paar\ t1\ t2 \approx^? Paar\ s1\ s2) \# xs, ys) \vdash [] \rightsquigarrow ((t1 \approx^? s1) \# (t2 \approx^? s2) \# xs, ys)$
 $\mid$
 $func-sred[intro!]: ((Func\ F\ t1 \approx^? Func\ F\ t2) \# xs, ys) \vdash [] \rightsquigarrow ((t1 \approx^? t2) \# xs, ys) \mid$
 $abst-aa-sred[intro!]: ((Abst\ a\ t1 \approx^? Abst\ a\ t2) \# xs, ys) \vdash [] \rightsquigarrow ((t1 \approx^? t2) \# xs, ys) \mid$
 $abst-ab-sred[intro!]: a \neq b \implies$
 $((Abst\ a\ t1 \approx^? Abst\ b\ t2) \# xs, ys) \vdash [] \rightsquigarrow ((t1 \approx^? swap\ [(a, b)]$
 $t2) \# xs, (a \#^? t2) \# ys) \mid$
 $atom-sred[intro!]: ((Atom\ a \approx^? Atom\ a) \# xs, ys) \vdash [] \rightsquigarrow (xs, ys) \mid$
 $susp-sred[intro!]: ((Susp\ pi1\ X \approx^? Susp\ pi2\ X) \# xs, ys)$
 $\vdash [] \rightsquigarrow (xs, (map\ (\lambda a. a \#^? Susp\ []\ X)\ (ds-list\ pi1\ pi2))) @ ys) \mid$
 $var-1-sred[intro!]: \neg(occurs\ X\ t) \implies ((Susp\ pi\ X \approx^? t) \# xs, ys)$
 $\vdash [(X, swap\ (rev\ pi)\ t)] \rightsquigarrow apply-subst\ [(X, swap\ (rev\ pi)\ t)]$
 $(xs, ys) \mid$
 $var-2-sred[intro!]: \neg(occurs\ X\ t) \implies ((t \approx^? Susp\ pi\ X) \# xs, ys)$
 $\vdash [(X, swap\ (rev\ pi)\ t)] \rightsquigarrow apply-subst\ [(X, swap\ (rev\ pi)\ t)]$
 (xs, ys)

inductive-cases *s-red-elims*:

$((Unit \approx^? Unit) \# xs, ys) \vdash [] \rightsquigarrow (xs, ys)$
 $((Paar\ t1\ t2 \approx^? Paar\ s1\ s2) \# xs, ys) \vdash [] \rightsquigarrow ((t1 \approx^? s1) \# (t2 \approx^? s2) \# xs, ys)$
 $((Func\ F\ t1 \approx^? Func\ F\ t2) \# xs, ys) \vdash [] \rightsquigarrow ((t1 \approx^? t2) \# xs, ys)$
 $((Abst\ a\ t1 \approx^? Abst\ a\ t2) \# xs, ys) \vdash [] \rightsquigarrow ((t1 \approx^? t2) \# xs, ys)$
 $((Abst\ a\ t1 \approx^? Abst\ b\ t2) \# xs, ys) \vdash [] \rightsquigarrow ((t1 \approx^? swap\ [(a, b)]\ t2) \# xs, (a \#^? t2) \# ys)$
 $((Atom\ a \approx^? Atom\ a) \# xs, ys) \vdash [] \rightsquigarrow (xs, ys)$
 $((Susp\ pi1\ X \approx^? Susp\ pi2\ X) \# xs, ys) \vdash [] \rightsquigarrow (xs, (map\ (\lambda a. a \#^? Susp\ []\ X)\ (ds-list\ pi1\ pi2))) @ ys)$
 $((Susp\ pi\ X \approx^? t) \# xs, ys) \vdash [(X, swap\ (rev\ pi)\ t)] \rightsquigarrow apply-subst\ [(X, swap\ (rev\ pi)\ t)]$
 (xs, ys)
 $((t \approx^? Susp\ pi\ X) \# xs, ys) \vdash [(X, swap\ (rev\ pi)\ t)] \rightsquigarrow apply-subst\ [(X, swap\ (rev\ pi)\ t)]$
 (xs, ys)

lemma *solution-weak*:

assumes $(nabla1, s) \in U\ P$

shows $(nabla1 \cup nabla3, s) \in U\ P$

using *assms*

unfolding *all-solutions-def* **using** *fresh-weak equ-weak* **by** *auto*

lemma *solution-comp-id*:

shows $((nabla, s) \in U\ P) = ((nabla, s \cdot []) \in U\ P)$ **and**

$((nabla, s) \in U\ P) = ((nabla, [] \cdot s) \in U\ P)$

unfolding *all-solutions-def* **by** *auto*

lemma *solutions-subst-assoc*:

$((nabla, s1 \cdot (s2 \cdot s3)) \in U P) = ((nabla, (s1 \cdot s2) \cdot s3) \in U P)$
unfolding *all-solutions-def* **using** *subst-assoc* **by** *simp*

inductive *c-red* :: *problem-type* $\Rightarrow$ *fresh-envs* $\Rightarrow$ *problem-type* $\Rightarrow$ *bool* ($- \vdash - \rightarrow -$ [80,80,80] 80)

where

unit-cred[intro!]: $([], (a \#? Unit) \# xs) \vdash \{\} \rightarrow ([], xs) \mid$
paar-cred[intro!]: $([], (a \#? Paar\ t1\ t2) \# xs) \vdash \{\} \rightarrow ([], (a \#? t1) \# (a \#? t2) \# xs) \mid$
func-cred[intro!]: $([], (a \#? Func\ F\ t) \# xs) \vdash \{\} \rightarrow ([], (a \#? t) \# xs) \mid$
abst-aa-cred[intro!]: $([], (a \#? Abst\ a\ t) \# xs) \vdash \{\} \rightarrow ([], xs) \mid$
abst-ab-cred[intro!]: $a \neq b \Rightarrow ([], (a \#? Abst\ b\ t) \# xs) \vdash \{\} \rightarrow ([], (a \#? t) \# xs) \mid$
atom-cred[intro!]: $a \neq b \Rightarrow ([], (a \#? Atom\ b) \# xs) \vdash \{\} \rightarrow ([], xs) \mid$
susp-cred[intro!]: $([], (a \#? Susp\ pi\ X) \# xs) \vdash \{((swapas\ (rev\ pi)\ a), X)\} \rightarrow ([], xs)$

inductive *red-plus* :: *problem-type* $\Rightarrow$ *unifier-type* $\Rightarrow$ *problem-type* $\Rightarrow$ *bool* ($- \models -$ $\Rightarrow -$ [80,80,80] 80)

where

sred-single[intro!]: $\llbracket P1 \vdash s1 \rightsquigarrow P2 \rrbracket \Rightarrow P1 \models (\{\}, s1) \Rightarrow P2 \mid$
cred-single[intro!]: $\llbracket P1 \vdash nabla1 \rightarrow P2 \rrbracket \Rightarrow P1 \models (nabla1, []) \Rightarrow P2 \mid$
sred-step[intro!]: $\llbracket P1 \vdash s1 \rightsquigarrow P2; P2 \models (nabla2, s2) \Rightarrow P3 \rrbracket \Rightarrow P1 \models (nabla2, (s2 \cdot s1)) \Rightarrow P3$
 $\mid$
cred-step[intro!]: $\llbracket P1 \vdash nabla1 \rightarrow P2; P2 \models (nabla2, []) \Rightarrow P3 \rrbracket \Rightarrow P1 \models (nabla2 \cup nabla1, []) \Rightarrow P3$

lemma *mgu-idem*:

assumes $(nabla1, s1) \in U P$
and $\forall (nabla2, s2) \in U P. nabla2 \models (subst\ s2)\ nabla1 \wedge nabla2 \models subst(s2 \cdot s1) \approx_{subst} s2$
shows $mgu\ P\ (nabla1, s1) \wedge idem\ (nabla1, s1)$
using *assms mgu-def idem-def* **by** *auto*

lemma *problem-subst-comm*: $((nabla, s2) \in U (apply_subst\ s1\ P)) = ((nabla, (s2 \cdot s1)) \in U P)$

using *all-solutions-def apply-subst-def subst-comp-expand* **by** *auto*

lemma *P1-to-P2-sred*:

assumes $(nabla1, s1) \in U P1$ **and** $P1 \vdash s2 \rightsquigarrow P2$
shows $(nabla1, s1) \in U P2 \wedge (nabla1 \models_{subst} (s1 \cdot s2) \approx_{subst} s1)$
using *assms*
proof(*induction arbitrary: nabla1 s1 rule: s-red.induct[OF P1 $\vdash$ s2 $\rightsquigarrow$ P2]*)
case (1 *xs ys*)

```

then have subst:  $\text{nabla}1 \models \text{subst } (s1 \cdot []) \approx \text{subst } s1$ 
  using equ-refl subst-equ-def by simp
from 1
have eqs:  $\forall (t1,t2) \in \text{set } xs. \text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  and fresh:  $\forall (a,t) \in \text{set } ys. \text{nabla}1 \vdash a \# \text{subst } s1 \ t$ 
  using all-solutions-def by simp-all
with subst show ?case
  using all-solutions-def by auto
next
case (2 t1 t2 t3 t4 xs ys)
then have subst:  $\text{nabla}1 \models \text{subst } (s1 \cdot []) \approx \text{subst } s1$ 
  using equ-refl subst-equ-def by simp

from 2
have fresh:  $\forall (a,t) \in \text{set } ys. \text{nabla}1 \vdash a \# \text{subst } s1 \ t$ 
  using all-solutions-def by simp

from 2
have i:  $\forall (t1,t2) \in \text{set } ((\text{Paar } t1 \ t2, \text{Paar } t3 \ t4) \# xs). \text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  using all-solutions-def by simp
hence  $\text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t3$ 
 $\text{nabla}1 \vdash \text{subst } s1 \ t2 \approx \text{subst } s1 \ t4$ 
  using Equ-elim(4)[of  $\text{nabla}1 \ \langle \text{subst } s1 \ t1 \rangle \ \langle \text{subst } s1 \ t2 \rangle \ \langle \text{subst } s1 \ t3 \rangle \ \langle \text{subst } s1 \ t4 \rangle$ ]
  subst-paar[of s1] by auto+
with i have eqs:
 $\forall (t1,t2) \in \text{set } ((t1,t3) \# (t2, t4) \# xs). \text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  by auto

from fresh eqs subst
show ?case using all-solutions-def by simp
next
case (3 F t1 t2 xs ys)
then have subst:  $\text{nabla}1 \models \text{subst } (s1 \cdot []) \approx \text{subst } s1$ 
  using equ-refl subst-equ-def by simp

from 3
have fresh:  $\forall (a,t) \in \text{set } ys. \text{nabla}1 \vdash a \# \text{subst } s1 \ t$ 
  using all-solutions-def by simp

from 3
have i:  $\forall (t1,t2) \in \text{set } ((\text{Func } F \ t1, \text{Func } F \ t2) \# xs). \text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  using all-solutions-def by simp
hence  $\text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  using Equ-elim(5)[of  $\text{nabla}1 \ F \ \langle \text{subst } s1 \ t1 \rangle \ \langle \text{subst } s1 \ t2 \rangle$ ]
  subst-func by auto
with i have eqs:

```

```

   $\forall (t1, t2) \in \text{set } ((t1, t2) \# xs). \text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  by auto

  from subst fresh eqs
  show ?case using all-solutions-def by simp
next
  case (4 a t1 t2 xs ys)
  then have subst:  $\text{nabla}1 \models \text{subst } (s1 \cdot []) \approx \text{subst } s1$ 
    using equ-refl subst-eq-def by simp

  from 4
  have fresh:  $\forall (a, t) \in \text{set } ys. \text{nabla}1 \vdash a \# \text{subst } s1 \ t$ 
    using all-solutions-def by simp

  from 4
  have i:  $\forall (t1, t2) \in \text{set } ((\text{Abst } a \ t1, \text{Abst } a \ t2) \# xs). \text{nabla}1 \vdash \text{subst } s1 \ t1 \approx$ 
 $\text{subst } s1 \ t2$ 
    using all-solutions-def by simp
  hence  $\text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
    using Equ-elim(6)[of  $\text{nabla}1 \ a \ \langle \text{subst } s1 \ t1 \rangle \ \langle \text{subst } s1 \ t2 \rangle$ ]
    subst-abst by auto
  with i have eqs:
     $\forall (t1, t2) \in \text{set } ((t1, t2) \# xs). \text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
    by auto

  from subst fresh eqs
  show ?case using all-solutions-def by simp
next
  case (5 a b t1 t2 xs ys)
  then have subst:  $\text{nabla}1 \models \text{subst } (s1 \cdot []) \approx \text{subst } s1$ 
    using equ-refl subst-eq-def by simp

  from 5
  have eqsP1:  $\forall (t1, t2) \in \text{set } ((\text{Abst } a \ t1, \text{Abst } b \ t2) \# xs). \text{nabla}1 \vdash \text{subst } s1 \ t1$ 
 $\approx \text{subst } s1 \ t2$ 
    using all-solutions-def by simp
  hence i:  $\text{nabla}1 \vdash \text{Abst } a \ (\text{subst } s1 \ t1) \approx \text{Abst } b \ (\text{subst } s1 \ t2)$ 
    using subst-abst[of s1] by simp

  from 5
  have ii:  $\forall (a, t) \in \text{set } ys. \text{nabla}1 \vdash a \# \text{subst } s1 \ t$ 
    using all-solutions-def by auto
  from 5 i
  have  $\text{nabla}1 \vdash a \# \text{subst } s1 \ t2$ 
    using Equ-elim(7) by blast
  with ii
  have fresh:  $\forall (a, t) \in \text{set } ((a, t2) \# ys). \text{nabla}1 \vdash a \# \text{subst } s1 \ t$ 
    by simp

```

```

from  $i$  5
have  $nabla1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ (\text{swap } [(a, b)] \ t2)$ 
  using 5 Equ-elim(7)[of  $nabla1 \ a \ \langle \text{subst } s1 \ t1 \rangle \ b \ \langle \text{subst } s1 \ t2 \rangle$ ]
  subst-swap-comm by simp
with eqsP1 have eqs:
   $\forall (t1, t2) \in \text{set } ((t1, \text{swap } [(a, b)] \ t2) \# \text{xs}). \nabla1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  by auto

from subst fresh eqs
show ?case using all-solutions-def by simp
next
case (6  $a \ \text{xs} \ \text{ys}$ )
then have subst:  $nabla1 \models \text{subst } (s1 \cdot []) \approx \text{subst } s1$ 
  using equ-refl subst-equ-def by simp

from 6
have fresh:  $\forall (a, t) \in \text{set } \text{ys}. \nabla1 \vdash a \# \text{subst } s1 \ t$ 
  using all-solutions-def by auto

from 6
have eqs:  $\forall (t1, t2) \in \text{set } \text{xs}. \nabla1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  using all-solutions-def by auto

from subst fresh eqs
show ?case using all-solutions-def by simp
next
case (7  $\pi1 \ X \ \pi2 \ \text{xs} \ \text{ys}$ )
then have subst:  $nabla1 \models \text{subst } (s1 \cdot []) \approx \text{subst } s1$ 
  using equ-refl subst-equ-def by simp

from 7
have  $i$ :  $\forall (t1, t2) \in \text{set } ((\text{Susp } \pi1 \ X, \text{Susp } \pi2 \ X) \# \text{xs}). \nabla1 \vdash \text{subst } s1 \ t1$ 
 $\approx \text{subst } s1 \ t2$ 
   $\forall (a, t) \in \text{set } \text{ys}. \nabla1 \vdash a \# \text{subst } s1 \ t$ 
  using all-solutions-def by simp +

from 7(1)
have  $nabla1 \vdash \text{swap } \pi1 \ (\text{look-up } X \ s1) \approx \text{swap } \pi2 \ (\text{look-up } X \ s1)$ 
  using all-solutions-def subst-susp by simp
hence  $\forall a \in \text{ds } \pi1 \ \pi2. \nabla1 \vdash a \# (\text{look-up } X \ s1)$ 
  using equ- $\pi1$ - $\pi2$ -dec by simp
hence  $\forall a \in \text{set } (\text{ds-list } \pi1 \ \pi2). \nabla1 \vdash a \# \text{subst } s1 \ (\text{Susp } [] \ X)$ 
  using subst-susp ds-list-equ-ds[of  $\pi1 \ \pi2$ ] by simp
hence  $\forall (a, t) \in \text{set } (\text{map } (\lambda a. a \#? \text{Susp } [] \ X) \ (\text{ds-list } \pi1 \ \pi2)). \nabla1 \vdash a \#$ 
 $\text{subst } s1 \ t$ 
  by (auto split: prod.splits)
with 7  $i(2)$ 
have fresh:

```

```

       $\forall (a,t) \in \text{set } (\text{map } (\lambda a. (a, \text{Susp } [] X)) (\text{ds-list } \pi_1 \pi_2) @ \text{ys}). \text{nabla}1 \vdash a \#$ 
subst s1 t
    by auto

from i(1)
have eqs:
   $\forall (t1,t2) \in \text{set } xs. \text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  by simp

from subst fresh eqs
show ?case using all-solutions-def by simp
next
case (8 X t' pi xs ys)
hence nabla1  $\vdash \text{subst } s1 \ (\text{Susp } \pi X) \approx \text{subst } s1 \ t'$ 
  using all-solutions-def[of  $\langle (\text{Susp } \pi X, t') \# xs, \text{ys} \rangle$ ] by auto
hence subst: nabla1  $\models \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \approx \text{subst } s1$ 
  using unif-1 by simp

from 8
have eqs-old:
   $\forall (t1,t2) \in \text{set } xs. \text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  and fresh-old:
   $\forall (a,t) \in \text{set } \text{ys}. \text{nabla}1 \vdash a \# \text{subst } s1 \ t$ 
  using all-solutions-def by auto

from eqs-old
have eqs:  $\forall (t1,t2) \in \text{set } xs.$ 
  nabla1  $\vdash \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ t1 \approx \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ t2$ 
  using unif-2a[OF subst] by auto

from fresh-old
have fresh:  $\forall (a,t) \in \text{set } \text{ys}. \text{nabla}1 \vdash a \# \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ t$ 
  using unif-2b[OF subst] by auto

from eqs fresh
have (nabla1, (s1  $\cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \in U \ (xs, \text{ys})$ )
  using all-solutions-def by simp
hence U: (nabla1, s1)  $\in U \ (\text{apply-subst } [(X, \text{swap } (\text{rev } \pi) \ t')] \ (xs, \text{ys}))$ 
  using problem-subst-comm by simp

from subst U
show ?case by simp
next
case (9 X t' pi xs ys)
hence nabla1  $\vdash \text{subst } s1 \ t' \approx \text{subst } s1 \ (\text{Susp } \pi X)$ 
  using all-solutions-def[of  $\langle (t', \text{Susp } \pi X) \# xs, \text{ys} \rangle$ ] by simp
hence subst: nabla1  $\models \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \approx \text{subst } s1$ 
  using unif-1[OF equ-symm] by blast

```

```

from  $g$ 
have eqs-old:
   $\forall (t1, t2) \in \text{set } xs. \text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  and fresh-old:
   $\forall (a, t) \in \text{set } ys. \text{nabla}1 \vdash a \# \text{subst } s1 \ t$ 
  using all-solutions-def by auto

from eqs-old
have eqs:  $\forall (t1, t2) \in \text{set } xs.$ 
   $\text{nabla}1 \vdash \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ t1 \approx \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ t2$ 
  using unif-2a[OF subst] by auto

from fresh-old
have fresh:  $\forall (a, t) \in \text{set } ys. \text{nabla}1 \vdash a \# \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ t$ 
  using unif-2b[OF subst] by auto

from eqs fresh
have  $(\text{nabla}1, (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')])) \in U \ (xs, ys)$ 
  using all-solutions-def by simp
hence  $U: (\text{nabla}1, s1) \in U \ (\text{apply-subst } [(X, \text{swap } (\text{rev } \pi) \ t')] \ (xs, ys))$ 
  using problem-subst-comm by simp

from subst U
show ?case by simp
qed

lemma P1-from-P2-sred:
  assumes  $(\text{nabla}1, s1) \in U \ P2$  and  $P1 \vdash s2 \rightsquigarrow P2$ 
  shows  $(\text{nabla}1, s1 \cdot s2) \in U \ P1$ 
  using assms
proof(induction arbitrary: nabla1 s1 rule: s-red.induct[OF  $\langle P1 \vdash s2 \rightsquigarrow P2 \rangle$ ])
  case (1 xs ys)
  hence  $\forall (t1, t2) \in \text{set } xs. \text{nabla}1 \vdash \text{subst } (s1 \cdot []) \ t1 \approx \text{subst } (s1 \cdot []) \ t2$ 
  and fresh:  $\forall (a, t) \in \text{set } ys. \text{nabla}1 \vdash a \# \text{subst } (s1 \cdot []) \ t$ 
  using all-solutions-def by simp-all
  hence eqs:
     $\forall (t1, t2) \in \text{set } ((\text{Unit}, \text{Unit}) \# xs). \text{nabla}1 \vdash \text{subst } (s1 \cdot []) \ t1 \approx \text{subst } (s1 \cdot []) \ t2$ 
  using subst-unit[of  $\langle (s1 \cdot []) \rangle$ ] equ-refl[of nabla1 Unit] by auto
  from fresh eqs show ?case
  using all-solutions-def by simp
next
  case (2 t1 t2 t3 t4 xs ys)
  hence eqs-old:  $\forall (t1, t2) \in \text{set } ((t1, t3) \# (t2, t4) \# xs). \text{nabla}1 \vdash \text{subst } (s1 \cdot []) \ t1 \approx \text{subst } (s1 \cdot []) \ t2$ 

```



```

    and fresh:  $\forall (a,t) \in \text{set } ys. \text{nabla}1 \vdash a \# \text{subst } (s1 \cdot []) t$ 
    using all-solutions-def by simp-all
  hence  $\text{nabla}1 \vdash \text{subst } (s1 \cdot []) t1 \approx \text{subst } (s1 \cdot []) t3$ 
     $\text{nabla}1 \vdash \text{subst } (s1 \cdot []) t2 \approx \text{subst } (s1 \cdot []) t4$  by simp-all
  hence  $\text{nabla}1 \vdash \text{subst } (s1 \cdot []) (\text{Paar } t1 \ t2) \approx \text{subst } (s1 \cdot []) (\text{Paar } t3 \ t4)$ 
    using equ-paar by simp
  with eqs-old have eqs:
     $\forall (t1,t2) \in \text{set } ((\text{Paar } t1 \ t2, \text{Paar } t3 \ t4) \# xs). \text{nabla}1 \vdash \text{subst } (s1 \cdot []) t1 \approx$ 
 $\text{subst } (s1 \cdot []) t2$ 
    by auto
  from fresh eqs show ?case
    using all-solutions-def by simp
next
  case (3 f t1 t2 xs ys)
  hence eqs-old:  $\forall (t1,t2) \in \text{set } ((t1, t2) \# xs). \text{nabla}1 \vdash \text{subst } (s1 \cdot []) t1 \approx$ 
 $\text{subst } (s1 \cdot []) t2$ 
    and fresh:  $\forall (a,t) \in \text{set } ys. \text{nabla}1 \vdash a \# \text{subst } (s1 \cdot []) t$ 
    using all-solutions-def by simp-all
  hence  $\text{nabla}1 \vdash \text{subst } (s1 \cdot []) (\text{Func } f \ t1) \approx \text{subst } (s1 \cdot []) (\text{Func } f \ t2)$ 
    using equ-func by simp
  with eqs-old have eqs:
     $\forall (t1,t2) \in \text{set } ((\text{Func } f \ t1, \text{Func } f \ t2) \# xs). \text{nabla}1 \vdash \text{subst } (s1 \cdot []) t1 \approx$ 
 $\text{subst } (s1 \cdot []) t2$ 
    by auto
  from fresh eqs show ?case
    using all-solutions-def by simp
next
  case (4 a t1 t2 xs ys)
  hence eqs-old:  $\forall (t1,t2) \in \text{set } ((t1, t2) \# xs). \text{nabla}1 \vdash \text{subst } (s1 \cdot []) t1 \approx$ 
 $\text{subst } (s1 \cdot []) t2$ 
    and fresh:  $\forall (a,t) \in \text{set } ys. \text{nabla}1 \vdash a \# \text{subst } (s1 \cdot []) t$ 
    using all-solutions-def by simp-all
  hence  $\text{nabla}1 \vdash \text{subst } (s1 \cdot []) (\text{Abst } a \ t1) \approx \text{subst } (s1 \cdot []) (\text{Abst } a \ t2)$ 
    using equ-abst-aa by simp
  with eqs-old have eqs:
     $\forall (t1,t2) \in \text{set } ((\text{Abst } a \ t1, \text{Abst } a \ t2) \# xs). \text{nabla}1 \vdash \text{subst } (s1 \cdot []) t1 \approx$ 
 $\text{subst } (s1 \cdot []) t2$ 
    by auto
  from fresh eqs show ?case
    using all-solutions-def by simp
next
  case (5 a b t1 t2 xs ys)
  hence eqs-old:  $\forall (t1,t2) \in \text{set } ((t1, \text{swap } [(a, b)] \ t2) \# xs). \text{nabla}1 \vdash \text{subst } (s1$ 
 $\cdot []) t1 \approx \text{subst } (s1 \cdot []) t2$ 
    and fresh:  $\forall (a,t) \in \text{set } ((a, t2) \# ys). \text{nabla}1 \vdash a \# \text{subst } (s1 \cdot []) t$ 
    using all-solutions-def by simp-all
  hence  $\text{nabla}1 \vdash \text{subst } (s1 \cdot []) t1 \approx \text{swap } [(a, b)] (\text{subst } (s1 \cdot []) t2)$ 
     $\text{nabla}1 \vdash a \# \text{subst } (s1 \cdot []) t2$ 
    using subst-swap-comm[of  $\langle (s1 \cdot []) \rangle \ \langle [(a, b)] \rangle \ t2]$  by simp-all

```

```

hence  $\text{nabla}1 \vdash \text{subst } (s1 \cdot []) (Abst \ a \ t1) \approx \text{subst } (s1 \cdot []) (Abst \ b \ t2)$ 
using  $\text{equ-abst-ab}[OF \ 5(1) \ \langle \text{nabla}1 \vdash a \# \text{subst } (s1 \cdot []) \ t2 \rangle]$  subst-abst by simp
with eqs-old have eqs:
   $\forall (t1, t2) \in \text{set } ((Abst \ a \ t1, \ Abst \ b \ t2) \# \ xs). \ \text{nabla}1 \vdash \text{subst } (s1 \cdot []) \ t1 \approx$ 
 $\text{subst } (s1 \cdot []) \ t2$ 
  by auto
from fresh eqs show ?case
using all-solutions-def by simp
next
case (6 a xs ys)
hence  $\forall (t1, t2) \in \text{set } xs. \ \text{nabla}1 \vdash \text{subst } (s1 \cdot []) \ t1 \approx \text{subst } (s1 \cdot []) \ t2$ 
and fresh:  $\forall (a, t) \in \text{set } ys. \ \text{nabla}1 \vdash a \# \text{subst } (s1 \cdot []) \ t$ 
using all-solutions-def by simp-all
hence eqs:
   $\forall (t1, t2) \in \text{set } ((Atom \ a, \ Atom \ a) \# \ xs). \ \text{nabla}1 \vdash \text{subst } (s1 \cdot []) \ t1 \approx \text{subst}$ 
 $(s1 \cdot []) \ t2$ 
using subst-unit[of  $\langle (s1 \cdot []) \rangle$ ] equ-refl[of nabla1 Unit] by auto
from fresh eqs show ?case
using all-solutions-def by simp
next
case (7 pi1 X pi2 xs ys)
hence eqs-old:  $\forall (t1, t2) \in \text{set } xs. \ \text{nabla}1 \vdash \text{subst } (s1 \cdot []) \ t1 \approx \text{subst } (s1 \cdot []) \ t2$ 
and fresh-old:
   $\forall (a, t) \in \text{set } (\text{map } (\lambda a. (a, \text{Susp } [] \ X)) \ (\text{ds-list } pi1 \ pi2) \ @ \ ys). \ \text{nabla}1 \vdash a \#$ 
 $\text{subst } (s1 \cdot []) \ t$ 
using all-solutions-def by simp-all
hence  $\forall c \in \text{set } (\text{ds-list } pi1 \ pi2). \ \text{nabla}1 \vdash c \# \text{subst } (s1 \cdot []) \ (\text{Susp } [] \ X)$ 
by (auto split: prod.splits)
hence  $\forall c \in \text{ds } pi1 \ pi2. \ \text{nabla}1 \vdash c \# \text{subst } (s1 \cdot []) \ (\text{Susp } [] \ X)$ 
using ds-list-equ-ds by simp
hence  $\forall c \in \text{ds } pi1 \ pi2. \ \text{nabla}1 \vdash c \# (\text{look-up } X \ (s1 \cdot []))$ 
using subst-susp[of  $\langle (s1 \cdot []) \rangle \ \langle [] \rangle \ X$ ] swap-id by simp
hence  $\text{nabla}1 \vdash \text{subst } (s1 \cdot []) \ (\text{Susp } pi1 \ X) \approx \text{subst } (s1 \cdot []) \ (\text{Susp } pi2 \ X)$ 
using equ-equiv-pi subst-susp[of  $\langle (s1 \cdot []) \rangle \ - \ X$ ] by auto
with eqs-old fresh-old have
eqs:  $\forall (t1, t2) \in \text{set } ((\text{Susp } pi1 \ X, \ \text{Susp } pi2 \ X) \# \ xs).$ 
 $\text{nabla}1 \vdash \text{subst } (s1 \cdot []) \ t1 \approx \text{subst } (s1 \cdot []) \ t2$ 
and fresh:  $\forall (a, t) \in \text{set } ys. \ \text{nabla}1 \vdash a \# \text{subst } (s1 \cdot []) \ t$ 
by simp-all
then show ?case
using all-solutions-def by simp
next
case (8 X t' pi xs ys)
hence  $(\text{nabla}1, \ s1 \cdot [(X, \ \text{swap } (\text{rev } pi) \ t')]) \in U \ (xs, \ ys)$ 
using problem-subst-comm by simp
hence eqs-old:
   $\forall (t1, t2) \in \text{set } xs.$ 
 $\text{nabla}1 \vdash \text{subst } (s1 \cdot [(X, \ \text{swap } (\text{rev } pi) \ t')]) \ t1 \approx \text{subst } (s1 \cdot [(X, \ \text{swap } (\text{rev } pi)$ 
 $t')]) \ t2$ 

```

and fresh: $\forall (a, t) \in \text{set } ys. \text{nabla}1 \vdash a \# \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ t$
using *all-solutions-def* **by** *simp-all*

have $\text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ (\text{Susp } \pi \ X) = \text{subst } s1 \ (\text{swap } \pi \ (\text{swap } (\text{rev } \pi) \ t'))$
using *subst-susp subst-swap-comm subst-comp-expand eq-fst-iff look-up.simps(2)*
snd-eqD **by** *metis*

also have $\dots = \text{swap } \pi \ (\text{swap } (\text{rev } \pi) \ (\text{subst } s1 \ t'))$
using *subst-swap-comm* **by** *simp*

hence $\text{nabla}1 \vdash \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ (\text{Susp } \pi \ X) \approx \text{subst } s1 \ t'$
using *equ-refl calculation rev-pi-pi-equ[of nabla1 pi ⟨subst s1 t'⟩]*
equ-trans[of nabla1 ⟨subst (s1 · [(X, swap (rev pi) t')]) (Susp pi X)⟩
⟨swap (rev pi) (swap pi (subst s1 t'))⟩ ⟨subst s1 t'⟩]
swap-inv-side **by** *auto*

with 8 eqs-old

have *eqs:* $\forall (t1, t2) \in \text{set } ((\text{Susp } \pi \ X, t') \# xs).$
 $\text{nabla}1 \vdash \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ t1 \approx \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ t2$
using *subst-comp-expand subst-not-occurs* **by** *simp*

from fresh eqs show ?case
using *all-solutions-def* **by** *simp*

next

case $(9 \ X \ t' \ \pi \ xs \ ys)$
hence $(\text{nabla}1, s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \in U \ (xs, ys)$
using *problem-subst-comm* **by** *simp*

hence *eqs-old:*
 $\forall (t1, t2) \in \text{set } xs.$
 $\text{nabla}1 \vdash \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ t1 \approx \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ t2$
and fresh: $\forall (a, t) \in \text{set } ys. \text{nabla}1 \vdash a \# \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ t$
using *all-solutions-def* **by** *simp-all*

have $\text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ (\text{Susp } \pi \ X) = \text{subst } s1 \ (\text{swap } \pi \ (\text{swap } (\text{rev } \pi) \ t'))$
using *subst-susp subst-swap-comm subst-comp-expand eq-fst-iff look-up.simps(2)*
snd-eqD **by** *metis*

also have $\dots = \text{swap } \pi \ (\text{swap } (\text{rev } \pi) \ (\text{subst } s1 \ t'))$
using *subst-swap-comm* **by** *simp*

hence $\text{nabla}1 \vdash \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ (\text{Susp } \pi \ X) \approx \text{subst } s1 \ t'$
using *equ-refl calculation rev-pi-pi-equ[of nabla1 pi ⟨subst s1 t'⟩]*
equ-trans[of nabla1 ⟨subst (s1 · [(X, swap (rev pi) t')]) (Susp pi X)⟩
⟨swap (rev pi) (swap pi (subst s1 t'))⟩ ⟨subst s1 t'⟩]
swap-inv-side **by** *auto*

hence $\text{nabla}1 \vdash \text{subst } s1 \ t' \approx \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) \ t')]) \ (\text{Susp } \pi \ X)$
using *equ-symm* **by** *simp*

with 9 eqs-old

have eqs: $\forall (t1, t2) \in \text{set } ((t', \text{Susp } \pi X) \# xs).$
 $\text{nabla}1 \vdash \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) t')]) t1 \approx \text{subst } (s1 \cdot [(X, \text{swap } (\text{rev } \pi) t')]) t2$
using *subst-comp-expand subst-not-occurs* **by** *simp*

from *fresh eqs* **show** *?case*
using *all-solutions-def* **by** *simp*
qed

lemma *P1-to-P2-cred*:
assumes $(\text{nabla}1, s1) \in U P1$
and $P1 \vdash \text{nabla}2 \rightarrow P2$
shows $(\text{nabla}1, s1) \in U P2 \wedge (\text{nabla}1 \models (\text{subst } s1) \text{nabla}2)$
using *assms*
proof(*induction arbitrary: nabla1 s1 rule: c-red.induct[OF P1 nabla2 $\rightarrow$ P2]*)
case (2 a t1 t2 ys)
hence *fresh-old*: $\forall (a, t) \in \text{set } ((a, \text{Paar } t1 t2) \# ys). \text{nabla}1 \vdash a \# \text{subst } s1 t$
and *eqs*: $\forall (t1, t2) \in \text{set } []. \text{nabla}1 \vdash \text{subst } s1 t1 \approx \text{subst } s1 t2$
using *all-solutions-def* **by** *simp-all*
hence $\text{nabla}1 \vdash a \# \text{subst } s1 (\text{Paar } t1 t2)$
by *simp*
hence $\text{nabla}1 \vdash a \# \text{subst } s1 t1 \text{ nabla}1 \vdash a \# \text{subst } s1 t2$
using *Fresh-elim5*(5)[*of nabla1 a <subst s1 t1>*
<subst s1 t2>] **by** *auto*
with *fresh-old* **have**
fresh: $\forall (a, t) \in \text{set } ((a, t1) \# (a, t2) \# ys). \text{nabla}1 \vdash a \# \text{subst } s1 t$
by *auto*
from *fresh eqs*
show *?case*
using *all-solutions-def ext-subst-def* **by** *simp*
next
case (3 a f t ys)
hence *fresh-old*: $\forall (a, t) \in \text{set } ((a, \text{Func } f t) \# ys). \text{nabla}1 \vdash a \# \text{subst } s1 t$
and *eqs*: $\forall (t1, t2) \in \text{set } []. \text{nabla}1 \vdash \text{subst } s1 t1 \approx \text{subst } s1 t2$
using *all-solutions-def* **by** *simp-all*
hence $\text{nabla}1 \vdash a \# \text{subst } s1 (\text{Func } f t)$
by *simp*
hence $\text{nabla}1 \vdash a \# \text{subst } s1 t$
using *Fresh-elim6*(6) **by** *auto*
with *fresh-old* **have**
fresh: $\forall (a, t) \in \text{set } ((a, t) \# ys). \text{nabla}1 \vdash a \# \text{subst } s1 t$
by *auto*
from *fresh eqs*
show *?case*
using *all-solutions-def ext-subst-def* **by** *simp*
next
case (5 a b t t')
hence *fresh-old*: $\forall (a, t') \in \text{set } ((a, \text{Abst } b t) \# ys). \text{nabla}1 \vdash a \# \text{subst } s1 t'$

and eqs: $\forall (t1, t2) \in \text{set } [] . \text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$
using *all-solutions-def* **by** *simp-all*
hence $\text{nabla}1 \vdash a \# \text{subst } s1 \ (\text{Abst } b \ t)$
by *simp*
hence $\text{nabla}1 \vdash a \# \text{subst } s1 \ t$
using 5 *Fresh-elim*(1)[*of* $\text{nabla}1 \ a \ b \ \langle \text{subst } s1 \ t \rangle$] **by** *auto*
with *fresh-old* **have**
fresh: $\forall (a, t') \in \text{set } ((a, t) \# \text{ys}) . \text{nabla}1 \vdash a \# \text{subst } s1 \ t'$
by *auto*
from *fresh eqs*
show ?*case*
using *all-solutions-def ext-subst-def* **by** *simp*
next
case $(\gamma \ b \ \text{pi } X \ \text{ys})$
hence *fresh-old*: $\forall (a, t) \in \text{set } ((b, \text{Susp } \text{pi } X) \# \text{ys}) . \text{nabla}1 \vdash a \# \text{subst } s1 \ t$
and eqs: $\forall (t1, t2) \in \text{set } [] . \text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$
using *all-solutions-def* **by** *simp-all*
hence *fresh*: $\forall (a, t) \in \text{set } \text{ys} . \text{nabla}1 \vdash a \# \text{subst } s1 \ t$ **by** *simp*
from *fresh-old* **have**
 $\text{nabla}1 \vdash b \# \text{subst } s1 \ (\text{Susp } \text{pi } X)$ **by** *simp*
hence $\text{nabla}1 \vdash \text{swapas } (\text{rev } \text{pi}) \ b \# \text{subst } s1 \ (\text{Susp } [] \ X)$
using *fresh-swap-left subst-susp* **by** *simp*
hence *ext*: $\text{nabla}1 \models (\text{subst } s1) \ \{(\text{swapas } (\text{rev } \text{pi}) \ b, \ X)\}$
using *ext-subst-def* **by** *simp*
from *fresh eqs ext*
show ?*case* **using** *all-solutions-def* **by** *simp*
qed (*auto simp add: all-solutions-def ext-subst-def*)

lemma *P1-from-P2-cred*:

assumes $(\text{nabla}1, s1) \in U \ P2$
and $P1 \vdash \text{nabla}2 \rightarrow P2$ **and** $\text{nabla}3 \models (\text{subst } s1) \ \text{nabla}2$
shows $(\text{nabla}1 \cup \text{nabla}3, s1) \in U \ P1$
using *assms*
proof(*induction arbitrary: nabla1 s1 rule: c-red.induct[OF $\langle P1 \vdash \text{nabla}2 \rightarrow P2 \rangle$]*)
case $(2 \ a \ t1 \ t2 \ xs)$
hence *fresh-old*:
 $\forall (b, t) \in \text{set } ((a, t1) \# (a, t2) \# xs) . \text{nabla}1 \vdash b \# \text{subst } s1 \ t$
and $\forall (t1, t2) \in \text{set } [] . \text{nabla}1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$
using *all-solutions-def* **by** *auto*
hence *weak1*: $\forall (b, t) \in \text{set } xs . (\text{nabla}1 \cup \text{nabla}3) \vdash b \# \text{subst } s1 \ t$
 $\forall (t1, t2) \in \text{set } [] . (\text{nabla}1 \cup \text{nabla}3) \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$
using *fresh-weak* **by** *auto*
from *fresh-old* **have** $\text{nabla}1 \vdash a \# \text{subst } s1 \ t1$ $\text{nabla}1 \vdash a \# \text{subst } s1 \ t2$
by *simp+*
hence $\text{nabla}1 \vdash a \# \text{subst } s1 \ (\text{Paar } t1 \ t2)$
using *fresh-paar subst-paar* **by** *auto*
hence $(\text{nabla}1 \cup \text{nabla}3) \vdash a \# \text{subst } s1 \ (\text{Paar } t1 \ t2)$
using *fresh-weak* **by** *auto*

```

with weak1
have fresh:  $\forall (b,t) \in \text{set } ((a, \text{Paar } t1 \ t2) \# xs). (nabla1 \cup nabla3) \vdash b \# \text{subst } s1 \ t$ 
  by simp
with weak1(2) show ?case
  using all-solutions-def ext-subst-def by simp
next
case (3 a f t' xs)
hence fresh-old:
   $\forall (b,t) \in \text{set } ((a,t') \# xs). nabla1 \vdash b \# \text{subst } s1 \ t$ 
  and  $\forall (t1, t2) \in \text{set } []. nabla1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  using all-solutions-def by auto
hence weak1:  $\forall (b,t) \in \text{set } xs. (nabla1 \cup nabla3) \vdash b \# \text{subst } s1 \ t$ 
   $\forall (t1, t2) \in \text{set } []. (nabla1 \cup nabla3) \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  using fresh-weak by auto
from fresh-old have  $nabla1 \vdash a \# \text{subst } s1 \ t'$ 
  by simp
hence  $nabla1 \vdash a \# \text{subst } s1 \ (\text{Func } f \ t')$ 
  using fresh-paar subst-paar by auto
hence  $(nabla1 \cup nabla3) \vdash a \# \text{subst } s1 \ (\text{Func } f \ t')$ 
  using fresh-weak by auto
with weak1
have fresh:  $\forall (b,t) \in \text{set } ((a, \text{Func } f \ t') \# xs). (nabla1 \cup nabla3) \vdash b \# \text{subst } s1 \ t$ 
  by simp
with weak1(2) show ?case
  using all-solutions-def ext-subst-def by simp
next
case (5 a b t' xs)
hence fresh-old:
   $\forall (c,t) \in \text{set } ((a,t') \# xs). nabla1 \vdash c \# \text{subst } s1 \ t$ 
  and  $\forall (t1, t2) \in \text{set } []. nabla1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  using all-solutions-def by auto
hence weak1:  $\forall (c,t) \in \text{set } xs. (nabla1 \cup nabla3) \vdash c \# \text{subst } s1 \ t$ 
   $\forall (t1, t2) \in \text{set } []. (nabla1 \cup nabla3) \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
  using fresh-weak by auto
from fresh-old have  $nabla1 \vdash a \# \text{subst } s1 \ t'$ 
  by simp
hence  $nabla1 \vdash a \# \text{subst } s1 \ (\text{Abst } b \ t')$ 
using fresh-abst-ab[OF  $\langle nabla1 \vdash a \# \text{subst } s1 \ t' \rangle$  5(1)] subst-abst by auto
hence  $(nabla1 \cup nabla3) \vdash a \# \text{subst } s1 \ (\text{Abst } b \ t')$ 
  using fresh-weak by auto
with weak1
have fresh:  $\forall (b,t) \in \text{set } ((a, \text{Abst } b \ t') \# xs). (nabla1 \cup nabla3) \vdash b \# \text{subst } s1 \ t$ 
  by simp
with weak1(2) show ?case
  using all-solutions-def ext-subst-def by simp
next
case (7 b pi X xs)
hence fresh-old:  $\forall (a,t) \in \text{set } xs. nabla1 \vdash a \# \text{subst } s1 \ t$ 
  and  $\text{eqs: } \forall (t1, t2) \in \text{set } []. nabla1 \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 

```

```

    using all-solutions-def by simp-all
  hence weak1:  $\forall (a,t) \in \text{set } xs. (nabla1 \cup \nabla3) \vdash a \# \text{subst } s1 \ t$ 
     $\forall (t1, t2) \in \text{set } []. (\nabla1 \cup \nabla3) \vdash \text{subst } s1 \ t1 \approx \text{subst } s1 \ t2$ 
    using fresh-weak by auto
  from 7 have  $\nabla3 \vdash \text{swapas } (\text{rev } \pi) \ b \# \text{subst } s1 \ (\text{Susp } [] \ X)$ 
    using ext-subst-def by auto
  hence  $\nabla3 \vdash b \# \text{subst } s1 \ (\text{Susp } \pi \ X)$ 
    using fresh-swap-right subst-susp by auto
  hence weak2:  $(\nabla1 \cup \nabla3) \vdash b \# \text{subst } s1 \ (\text{Susp } \pi \ X)$ 
    using fresh-weak[of  $\nabla3 \ b \ - \ \nabla1$ ] Un-commute[of  $\nabla3 \ \nabla1$ ] by auto
  with weak1(1)
  have  $\forall (a,t) \in \text{set } ((b, \text{Susp } \pi \ X) \# xs). (\nabla1 \cup \nabla3) \vdash a \# \text{subst } s1 \ t$ 
    by simp
  with weak2 show ?case
    using ext-subst-def all-solutions-def by simp
qed (auto simp add: ext-subst-def all-solutions-def fresh-weak)

```

```

lemma P1-to-P2-red-plus1:
  assumes  $P1 \models (\nabla, s) \Rightarrow P2$ 
  and  $(\nabla1, s1) \in U \ P1$ 
  shows  $(\nabla1, s1) \in U \ P2$ 
  using assms
proof(induction  $P1 \equiv P1 \ \nabla \equiv (\nabla, s) \ P2 \equiv P2$  arbitrary:  $s \ \nabla1 \ \nabla \ s1 \ P1$ 
P2 rule: red-plus.induct)
  case (sred-single  $P1 \ s \ P2 \ \nabla1 \ s1$ )
  have  $P1 \vdash s \rightsquigarrow P2 \ (\nabla1, s1) \in U \ P1$  by fact+
  then show  $(\nabla1, s1) \in U \ P2$ 
    using P1-to-P2-sred by blast
next
  case (cred-single  $P1 \ \nabla1 \ P2 \ \nabla2 \ s1$ )
  have  $P1 \vdash \nabla1 \rightarrow P2 \ (\nabla2, s1) \in U \ P1$  by fact+
  then show  $(\nabla2, s1) \in U \ P2$ 
    using P1-to-P2-cred by blast
next
  case (sred-step  $P1 \ s \ P2 \ \nabla2 \ s2 \ P3 \ \nabla1 \ s1$ )
  have  $P1 \vdash s \rightsquigarrow P2$  and  $(\nabla1, s1) \in U \ P1$  by fact+
  then have  $(\nabla1, s1) \in U \ P2$  using P1-to-P2-sred by blast
  moreover
  have IH:  $(\nabla1, s1) \in U \ P2 \implies (\nabla1, s1) \in U \ P3$  by fact
  ultimately
  show  $(\nabla1, s1) \in U \ P3$  by simp
next
  case (cred-step  $P1 \ \nabla1 \ P2 \ \nabla2 \ P3 \ \nabla3 \ s1$ )
  have  $P1 \vdash \nabla1 \rightarrow P2$  and  $(\nabla3, s1) \in U \ P1$  by fact+
  then have  $(\nabla3, s1) \in U \ P2$  using P1-to-P2-cred by blast
  moreover

```

have $IH: (nabla3, s1) \in U P2 \implies (nabla3, s1) \in U P3$ **by** *fact*
 ultimately show $(nabla3, s1) \in U P3$ **by** *simp*
qed

lemma *P1-to-P2-red-plus3*:
 assumes $P1 \models (nabla, s) \Rightarrow P2$
 and $(nabla1, s1) \in U P1$
 shows $nabla1 \models (subst\ s1)\ nabla$
 using *assms*
proof(*induction* $P1 \equiv P1\ nabla \equiv (nabla, s)\ P2 \equiv P2$ *arbitrary: s nabla1 nabla s1 P1*
P2 rule: red-plus.induct)
 case (*sred-single* $P1\ s\ P2\ nabla1\ s1$)
 then show $nabla1 \models (subst\ s1)\ \{\}$ **by** (*simp add: ext-subst-def*)
next
 case (*cred-single* $P1\ nabla1\ P2\ nabla2\ s1$)
 have $P1 \vdash nabla1 \rightarrow P2\ (nabla2, s1) \in U P1$ **by** *fact+*
 then show $nabla2 \models (subst\ s1)\ nabla1$ **using** *P1-to-P2-cred* **by** *blast*
next
 case (*sred-step* $P1\ s\ P2\ nabla2\ s2\ P3\ nabla1\ s1$)
 have $P1 \vdash s \rightsquigarrow P2\ (nabla1, s1) \in U P1$ **by** *fact+*
 then have $(nabla1, s1) \in U P2$ **using** *P1-to-P2-sred* **by** *blast*
 moreover have $IH: (nabla1, s1) \in U P2 \implies nabla1 \models (subst\ s1)\ nabla2$ **by** *fact*
 ultimately show $nabla1 \models (subst\ s1)\ nabla2$ **by** *simp*
next
 case (*cred-step* $P1\ nabla1\ P2\ nabla2\ P3\ nabla3\ s1$)
 have $IH: (nabla3, s1) \in U P2 \implies nabla3 \models (subst\ s1)\ nabla2$ **by** *fact*
 have $as: P1 \vdash nabla1 \rightarrow P2\ (nabla3, s1) \in U P1$ **by** *fact+*

 from *as* have $nabla3 \models (subst\ s1)\ nabla1$ **using** *P1-to-P2-cred* **by** *blast*
 moreover
 from *as* have $(nabla3, s1) \in U P2$ **using** *P1-to-P2-cred* **by** *blast*
 then have $nabla3 \models (subst\ s1)\ nabla2$ **using** *IH* **by** *blast*
 ultimately show $nabla3 \models (subst\ s1)\ (nabla2 \cup nabla1)$
by(*auto simp add: ext-subst-def*)
qed

lemma *P1-to-P2-red-plus2*:
 assumes $P1 \models (nabla, s) \Rightarrow P2$
 and $(nabla1, s1) \in U P1$
 shows $nabla1 \models subst\ (s1 \cdot s) \approx subst\ s1$
 using *assms*
proof(*induction* $P1 \equiv P1\ nabla \equiv (nabla, s)\ P2 \equiv P2$ *arbitrary: s nabla1 nabla s1 P1*
P2 rule: red-plus.induct)
 case (*sred-single* $P1\ s\ P2\ nabla1\ s1$)
 have $P1 \vdash s \rightsquigarrow P2\ (nabla1, s1) \in U P1$ **by** *fact+*
 then show $nabla1 \models subst\ (s1 \cdot s) \approx subst\ s1$
using *P1-to-P2-sred* **by** *blast*
next


```

    case (cred-single P1 nabla1 P2 nabla2 s1)
    show nabla2  $\models$  subst (s1  $\cdot$  [])  $\approx_{\text{subst}}$  s1
      by (simp add: subst-equ-refl)
next
    case (sred-step P1 s P2 nabla2 s2 P3 nabla1 s1)
    have IH: (nabla1, s1)  $\in U$  P2  $\implies$  nabla1  $\models$  subst (s1  $\cdot$  s2)  $\approx_{\text{subst}}$  s1 by fact
    have as: P1  $\vdash s \rightsquigarrow$  P2 (nabla1, s1)  $\in U$  P1 by fact+

    from as have (nabla1, s1)  $\in U$  P2 using P1-to-P2-sred by blast
    with IH have nabla1  $\models$  subst (s1  $\cdot$  s2)  $\approx$  subst s1 by blast
    then have nabla1  $\models$  subst ((s1  $\cdot$  s2)  $\cdot$  s)  $\approx$  subst (s1  $\cdot$  s)
      by (simp add: subst-cancel-right)
    moreover
    from as have nabla1  $\models$  subst (s1  $\cdot$  s)  $\approx_{\text{subst}}$  s1
      using P1-to-P2-sred by blast
    ultimately
    show nabla1  $\models$  subst (s1  $\cdot$  (s2  $\cdot$  s))  $\approx$  subst s1
      using subst-assoc subst-trans by metis
next
    case (cred-step P1 nabla1 P2 nabla2 P3)
    show nabla1  $\models$  subst (s1  $\cdot$  [])  $\approx$  subst s1
      by (simp add: subst-equ-refl)
qed

lemma P1-from-P2-red-plus:
  assumes P1  $\models$  (nabla, s)  $\Rightarrow$  P2
  and (nabla1, s1)  $\in U$  P2 and nabla3  $\models$  (subst s1)(nabla)
  shows (nabla1  $\cup$  nabla3, (s1  $\cdot$  s))  $\in U$  P1
  using assms
proof(induction P1  $\equiv$  P1 nabla  $\equiv$  (nabla, s) P2  $\equiv$  P2 arbitrary: s nabla1 nabla
nabla3 s1 P1 P2 rule: red-plus.induct)
  case (sred-single P1 s P2 nabla1 nabla3)
  have P1  $\vdash s \rightsquigarrow$  P2 (nabla1, s1)  $\in U$  P2 by fact+
  then have (nabla1, s1  $\cdot$  s)  $\in U$  P1
    using P1-from-P2-sred by simp
  then show (nabla1  $\cup$  nabla3, s1  $\cdot$  s)  $\in U$  P1
    using P1-from-P2-sred solution-weak by simp
next
  case (cred-single P1 nabla P2 nabla1 nabla3)
  have P1  $\vdash$  nabla  $\rightarrow$  P2
    (nabla1, s1)  $\in U$  P2
    nabla3  $\models$  (subst s1) nabla by fact+
  hence (nabla1  $\cup$  nabla3, s1)  $\in U$  P1
    using P1-from-P2-cred by simp
  then show (nabla1  $\cup$  nabla3, s1  $\cdot$  [])  $\in U$  P1
    using solution-comp-id by auto
next
  case (sred-step P1 s P2 nabla2 s2 P3 nabla1 nabla3)
  have IH: (nabla1, s1)  $\in U$  P3  $\implies$ 

```

$nabla3 \models (\text{subst } s1) \nabla2 \implies (\nabla1 \cup \nabla3, s1 \cdot s2) \in U P2$ **by** *fact+*
have *as*: $(\nabla1, s1) \in U P3$
 $\nabla3 \models (\text{subst } s1) \nabla2$
 $P1 \vdash s \rightsquigarrow P2$ **by** *fact+*
hence $(\nabla1 \cup \nabla3, s1 \cdot s2) \in U P2$
using *IH* **by** *simp*
hence $(\nabla1 \cup \nabla3, (s1 \cdot s2) \cdot s) \in U P1$
using *as*(3)
by (*simp add: P1-from-P2-sred*)
then show $(\nabla1 \cup \nabla3, s1 \cdot (s2 \cdot s)) \in U P1$
using *solutions-subst-assoc* **by** *simp*
next
case (*cred-step* $P1 \nabla P2 \nabla3$)
have *IH*: $(\nabla1, s1) \in U P3 \implies$
 $\nabla3 \models (\text{subst } s1) \nabla2 \implies (\nabla1 \cup \nabla3, s1 \cdot []) \in U P2$ **by** *fact*
have *as*: $P1 \vdash \nabla \rightarrow P2$
 $(\nabla1, s1) \in U P3$
 $\nabla3 \models (\text{subst } s1) (\nabla2 \cup \nabla)$ **by** *fact+*
have *ext-substs*: $\nabla3 \models (\text{subst } s1) (\nabla2)$
 $\nabla3 \models (\text{subst } s1) \nabla$
using *ext-subst-strong[OF as(3)]* **by** *simp+*
hence *a*: $(\nabla1 \cup \nabla3, s1 \cdot []) \in U P2$
using *IH as(2)* **by** *simp*
hence $(\nabla1 \cup \nabla3 \cup \nabla3, s1 \cdot []) \in U P1$
using *as(1) ext-substs(2) P1-from-P2-cred solution-comp-id* **by** *blast*
then show $(\nabla1 \cup \nabla3, s1 \cdot []) \in U P1$
unfolding *all-solutions-def* **using** *Un-absorb* **by** *simp*
qed

lemma *mgu*:
assumes $P \models (\nabla, s) \Rightarrow ([], [])$
shows $\text{mgu } P (\nabla, s) \wedge \text{idem } (\nabla, s)$
proof(*rule mgu-idem*)
have *i*: $(\{\}, []) \in U ([], [])$
unfolding *all-solutions-def* **by** *simp*
then show $(\nabla, s) \in U P$
using *P1-from-P2-red-plus[OF asms i]*
 $\text{ext-subst-id solution-comp-id}(2)$ **by** *simp*

show $\forall (\nabla2, s2) \in U P. \nabla2 \models (\text{subst } s2) \nabla \wedge$
 $\nabla2 \models \text{subst } (s2 \cdot s) \approx \text{subst } s2$

proof
fix *x*
assume $x \in U P$
then show *case* *x* **of** $(\nabla2, s2) \Rightarrow$
 $\nabla2 \models (\text{subst } s2) \nabla \wedge$
 $\nabla2 \models \text{subst } (s2 \cdot s) \approx \text{subst } s2$

proof (*cases x*)
case (*Pair* $\nabla2 s2$)

```

hence  $(nabla2, s2) \in U P$ 
using  $\langle x \in U P \rangle$  by auto
hence  $nabla2 \models (subst\ s2)\ nabla \wedge nabla2 \models subst\ (s2 \cdot s) \approx subst\ s2$ 
using assms P1-to-P2-red-plus2 P1-to-P2-red-plus3 by auto+
with Pair show ?thesis by simp
qed
qed
qed

end

```

References

- [1] M. Ayala-Rincón, W. de Carvalho Segundo, M. Fernández, D. Nantes-Sobrinho, and A. C. R. Oliveira. A formalisation of nominal α -equivalence with a, c, and AC function symbols. *Theor. Comput. Sci.*, 781:3–23, 2019.
- [2] M. Ayala-Rincón, M. Fernández, and A. C. R. Oliveira. Completeness in PVS of a nominal unification algorithm. In M. R. F. Benevides and R. Thiemann, editors, *Proceedings of the Tenth Workshop on Logical and Semantic Frameworks, with Applications, LSFA 2015, Natal, Brazil, August 31 - September 1, 2015*, volume 323 of *Electronic Notes in Theoretical Computer Science*, pages 57–74. Elsevier, 2015.
- [3] W. L. R. de Carvalho. *Nominal Equational Problems Modulo Associativity, Commutativity and Associativity-Commutativity*. PhD thesis, Graduate Program in Informatics, University of Brasília, 2019. in English.
- [4] A. C. R. Oliveira. *Unification, Confluence, and Intersection Types for Nominal Rewriting Systems*. PhD thesis, Graduate Program in Informatics, University of Brasília, 2016. in English.
- [5] C. Urban. Nominal unification revisited. In M. Fernández, editor, *Proceedings 24th International Workshop on Unification, UNIF 2010, Edinburgh, United Kingdom, 14th July 2010*, volume 42 of *EPTCS*, pages 1–11, 2010.
- [6] C. Urban, A. M. Pitts, and M. Gabbay. Nominal unification. *Theor. Comput. Sci.*, 323(1-3):473–497, 2004.